Renewable Energy and Weather conditions

Objective

To estimate the change in energy consumption in 5 different weather regions.

In [1]:
```
# Libraries to help with reading and manipulating data
import numpy as np
import pandas as pd

# Libraries to help with data visualization
import matplotlib.pyplot as plt
import seaborn as sns

# split the data into train and test
from sklearn.model_selection import train_test_split

# to build linear regression_model using statsmodels
import statsmodels.api as sm

# to check model performance
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# to compute VIF
from statsmodels.stats.outliers_influence import variance_inflation_factor

#Ignore warnings
# To supress warnings
import warnings
warnings.filterwarnings("ignore")
```

In [2]:
```
data_df=pd.read_csv("solar_weather.csv", index_col=False)

#Maintain the original data
data=data_df.copy()
data.sample(10)
```

Out[2]:

	Time	Energy delta[Wh]	GHI	temp	pressure	humidity	wind_speed	rain_1h	snow_1h	clouds_all	isSun	su
45218	2018-04-22 00:30:00	0	0.0	4.9	1019	89	2.4	0.00	0.0	44	0	
106474	2020-01-21 02:30:00	0	0.0	1.7	1040	95	5.1	0.00	0.0	18	0	
121134	2020-06-21 19:30:00	0	0.0	15.9	1018	85	4.0	0.00	0.0	100	0	
164036	2021-09-11 17:00:00	30	2.5	20.8	1015	93	1.7	0.00	0.0	31	1	
72968	2019-02-05 02:00:00	0	0.0	0.1	1024	82	6.2	0.00	0.0	87	0	
161748	2021-08-18 21:00:00	0	0.0	16.8	1008	84	6.2	0.00	0.0	89	0	

	Time	Energy delta[Wh]	GHI	temp	pressure	humidity	wind_speed	rain_1h	snow_1h	clouds_all	isSun	su
21833	2017-08-21 10:15:00	499	57.8	15.2	1018	80	6.1	0.28	0.0	70	1	
14991	2017-06-11 03:45:00	130	6.3	10.6	1020	94	1.6	0.00	0.0	98	1	
71155	2019-01-17 04:45:00	0	0.0	2.7	996	88	7.0	0.36	0.0	100	0	
151833	2021-05-07 14:15:00	1469	88.7	9.5	1009	63	5.2	0.00	0.0	86	1	

Understanding the data

In [3]:
```
data.shape
```

Out[3]:
```
(196776, 17)
```

In [4]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 196776 entries, 0 to 196775
Data columns (total 17 columns):
 #   Column                  Non-Null Count   Dtype  
---  ------                  --------------   -----  
 0   Time                    196776 non-null  object 
 1   Energy delta[Wh]        196776 non-null  int64  
 2   GHI                     196776 non-null  float64
 3   temp                    196776 non-null  float64
 4   pressure                196776 non-null  int64  
 5   humidity                196776 non-null  int64  
 6   wind_speed              196776 non-null  float64
 7   rain_1h                 196776 non-null  float64
 8   snow_1h                 196776 non-null  float64
 9   clouds_all              196776 non-null  int64  
 10  isSun                   196776 non-null  int64  
 11  sunlightTime            196776 non-null  int64  
 12  dayLength               196776 non-null  int64  
 13  SunlightTime/daylength  196776 non-null  float64
 14  weather_type            196776 non-null  int64  
 15  hour                    196776 non-null  int64  
 16  month                   196776 non-null  int64  
dtypes: float64(6), int64(10), object(1)
memory usage: 25.5+ MB
```

In [5]:
```
data.describe()
```

Out[5]:

	Energy delta[Wh]	GHI	temp	pressure	humidity	wind_speed	rain_1h
count	196776.000000	196776.000000	196776.000000	196776.000000	196776.000000	196776.000000	196776.000000
mean	573.008228	32.596538	9.790521	1015.292780	79.810566	3.937746	0.066035
std	1044.824047	52.172018	7.995428	9.585773	15.604459	1.821694	0.278913
min	0.000000	0.000000	-16.600000	977.000000	22.000000	0.000000	0.000000
25%	0.000000	0.000000	3.600000	1010.000000	70.000000	2.600000	0.000000
50%	0.000000	1.600000	9.300000	1016.000000	84.000000	3.700000	0.000000
75%	577.000000	46.800000	15.700000	1021.000000	92.000000	5.000000	0.000000

	Energy delta[Wh]	GHI	temp	pressure	humidity	wind_speed	rain_1h
max	5020.000000	229.200000	35.800000	1047.000000	100.000000	14.300000	8.090000

In [6]:
```
data.isnull().sum()
```

Out[6]:
```
Time                      0
Energy delta[Wh]          0
GHI                       0
temp                      0
pressure                  0
humidity                  0
wind_speed                0
rain_1h                   0
snow_1h                   0
clouds_all                0
isSun                     0
sunlightTime              0
dayLength                 0
SunlightTime/daylength    0
weather_type              0
hour                      0
month                     0
dtype: int64
```

In [7]:
```
data.duplicated().sum()
```

Out[7]:
```
0
```

Observations

- There are no missing or duplicated values in the data

In [8]:
```
data.Time = data.Time.apply(pd.to_datetime)
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 196776 entries, 0 to 196775
Data columns (total 17 columns):
 #   Column                  Non-Null Count   Dtype
---  ------                  --------------   -----
 0   Time                    196776 non-null  datetime64[ns]
 1   Energy delta[Wh]        196776 non-null  int64
 2   GHI                     196776 non-null  float64
 3   temp                    196776 non-null  float64
 4   pressure                196776 non-null  int64
 5   humidity                196776 non-null  int64
 6   wind_speed              196776 non-null  float64
 7   rain_1h                 196776 non-null  float64
 8   snow_1h                 196776 non-null  float64
 9   clouds_all              196776 non-null  int64
 10  isSun                   196776 non-null  int64
 11  sunlightTime            196776 non-null  int64
 12  dayLength               196776 non-null  int64
 13  SunlightTime/daylength  196776 non-null  float64
 14  weather_type            196776 non-null  int64
 15  hour                    196776 non-null  int64
 16  month                   196776 non-null  int64
dtypes: datetime64[ns](1), float64(6), int64(10)
memory usage: 25.5 MB
```

In [9]:
```
data['capture_day'] = [d.dayofweek for d in data['Time']]
data['capture_year'] = [d.year for d in data['Time']]
data['Time'] = [d.time() for d in data['Time']]
data["capture_min"]=[d.minute for d in data["Time"]]
```

```
data.drop(["Time"], axis=1, inplace=True)
data.sample(10)
```

Out[9]:

	Energy delta[Wh]	GHI	temp	pressure	humidity	wind_speed	rain_1h	snow_1h	clouds_all	isSun	sunlightTime
26632	481	51.1	9.4	1012	84	4.4	0.0	0.0	79	1	300
164944	0	0.0	7.8	1025	99	1.8	0.0	0.0	100	0	0
145317	0	0.0	4.8	1034	89	2.5	0.0	0.0	97	0	0
45454	1231	84.3	14.7	1012	62	7.2	0.0	0.0	98	1	480
124242	195	4.6	14.1	1011	76	4.8	0.0	0.0	71	1	105
188433	0	0.0	17.5	1017	61	4.0	0.0	0.0	71	0	0
77061	0	0.0	4.6	1026	80	1.7	0.0	0.0	36	0	0
79003	0	0.0	2.4	1014	82	4.6	0.0	0.0	24	0	0
58722	174	16.4	22.4	1021	39	1.4	0.0	0.0	61	1	750
147468	0	0.0	4.8	1020	92	4.2	0.0	0.0	100	0	0

Exploratory Data Analysis

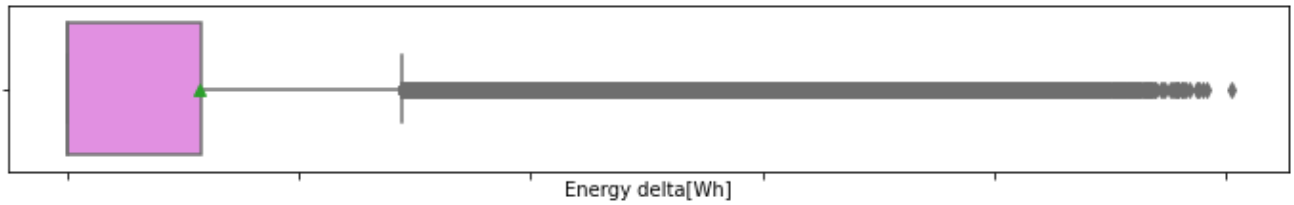
Univariate analysis

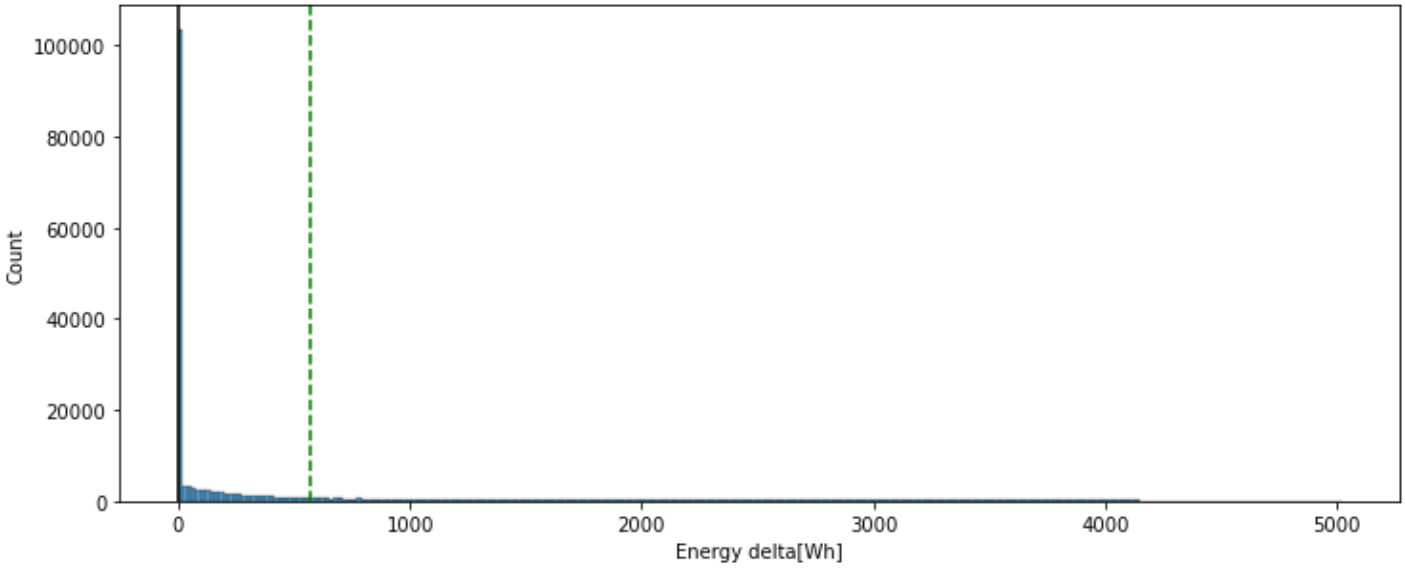
In [10]:

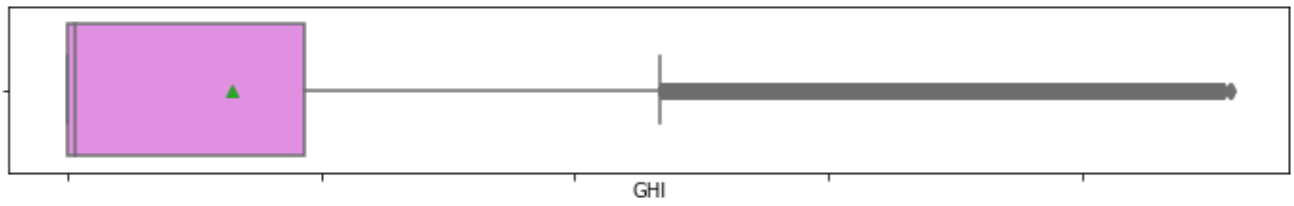
```
# function to plot a boxplot and a histogram along the same scale.


def histogram_boxplot(data, feature, figsize=(12, 7), kde=False, bins=None):
    f2, (ax_box2, ax_hist2) = plt.subplots(
        nrows=2,  # Number of rows of the subplot grid= 2
        sharex=True,  # x-axis will be shared among all subplots
        gridspec_kw={"height_ratios": (0.25, 0.75)},
        figsize=figsize,
    )  # creating the 2 subplots
    sns.boxplot(
        data=data, x=feature, ax=ax_box2, showmeans=True, color="violet"
    )  # boxplot will be created and a star will indicate the mean value of the column
    sns.histplot(
        data=data, x=feature, kde=kde, ax=ax_hist2, bins=bins, palette="winter"
    ) if bins else sns.histplot(
        data=data, x=feature, kde=kde, ax=ax_hist2
    )  # For histogram
    ax_hist2.axvline(
        data[feature].mean(), color="green", linestyle="--"
    )  # Add mean to the histogram
    ax_hist2.axvline(
        data[feature].median(), color="black", linestyle="-"
    )  # Add median to the histogram
```

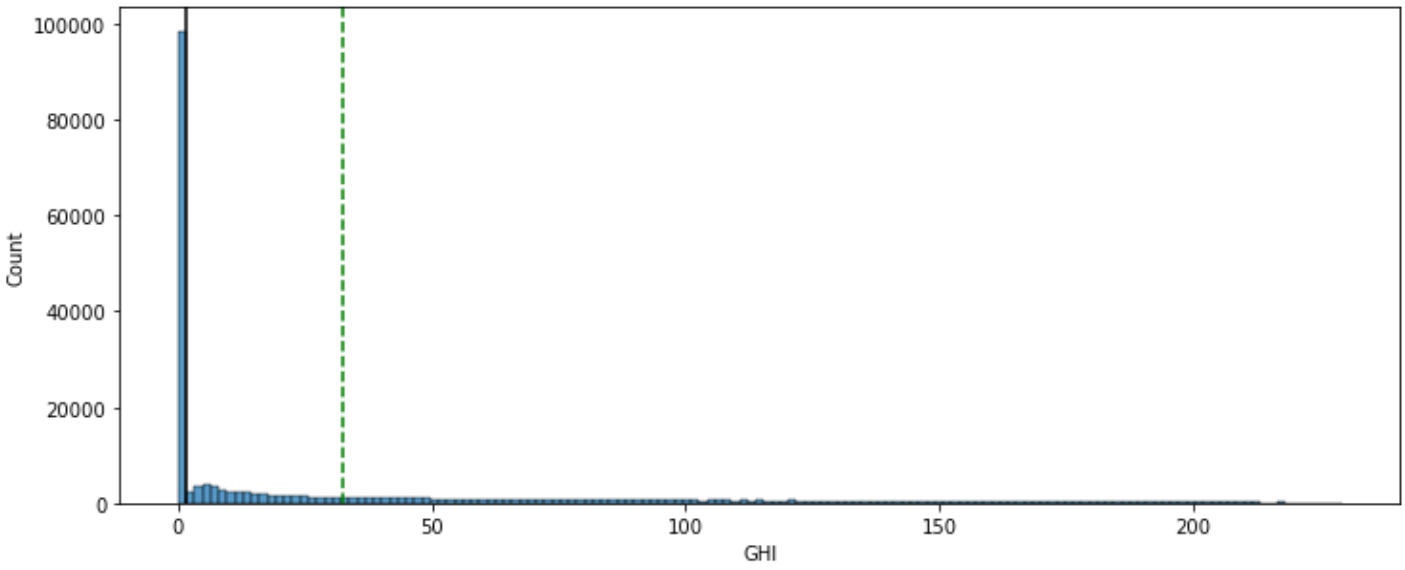
In [11]:

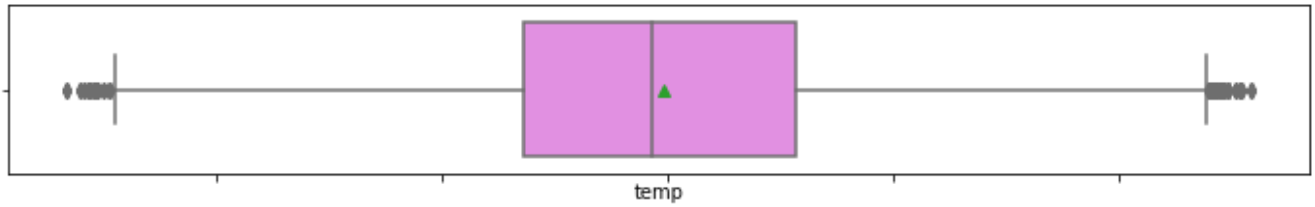
```
# selecting numerical columns
num_col = data.select_dtypes(include=np.number).columns.tolist()

for item in num_col:
    histogram_boxplot(data, item)
```

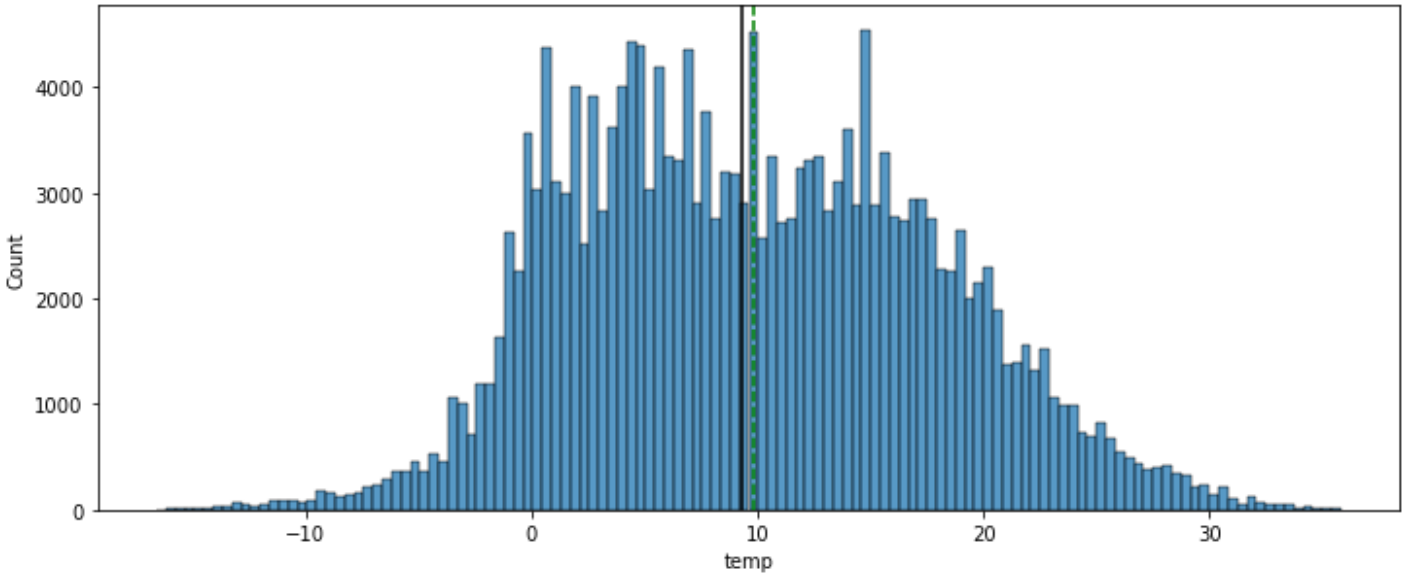


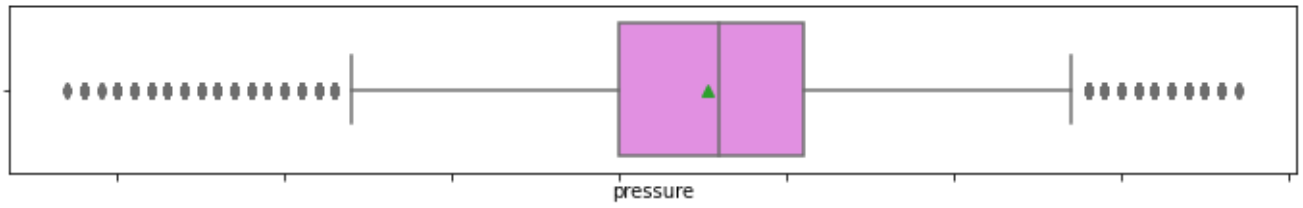


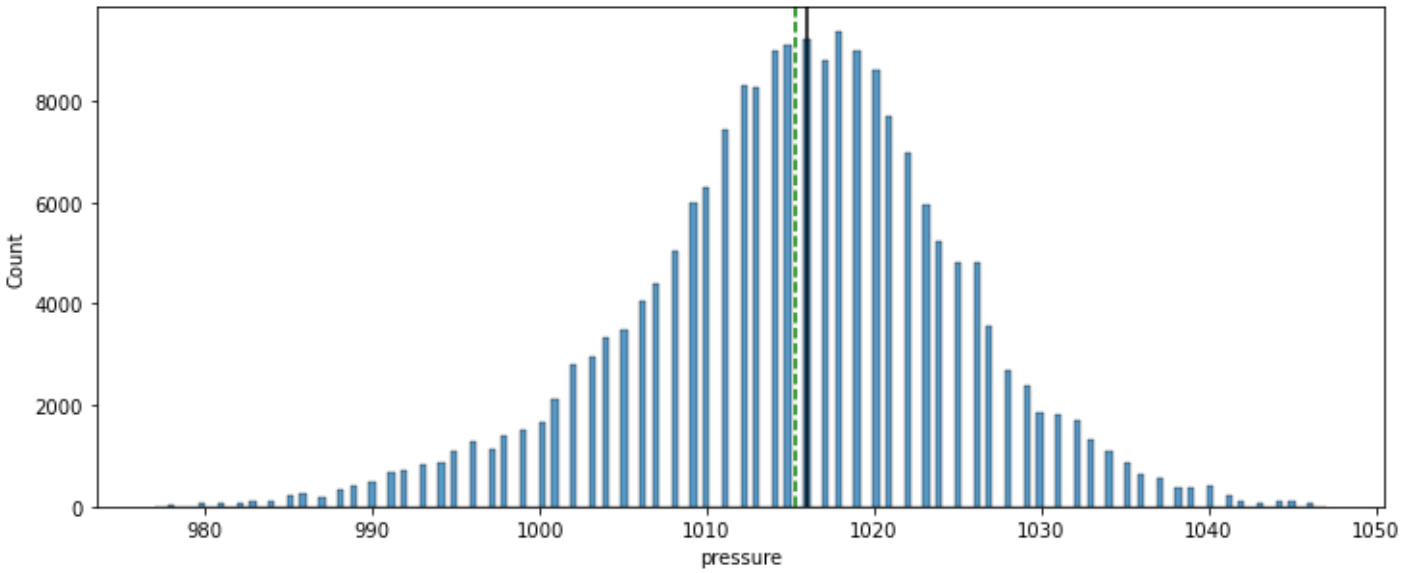


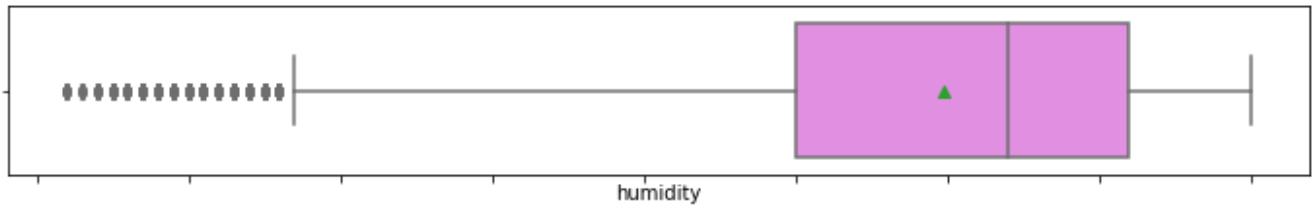


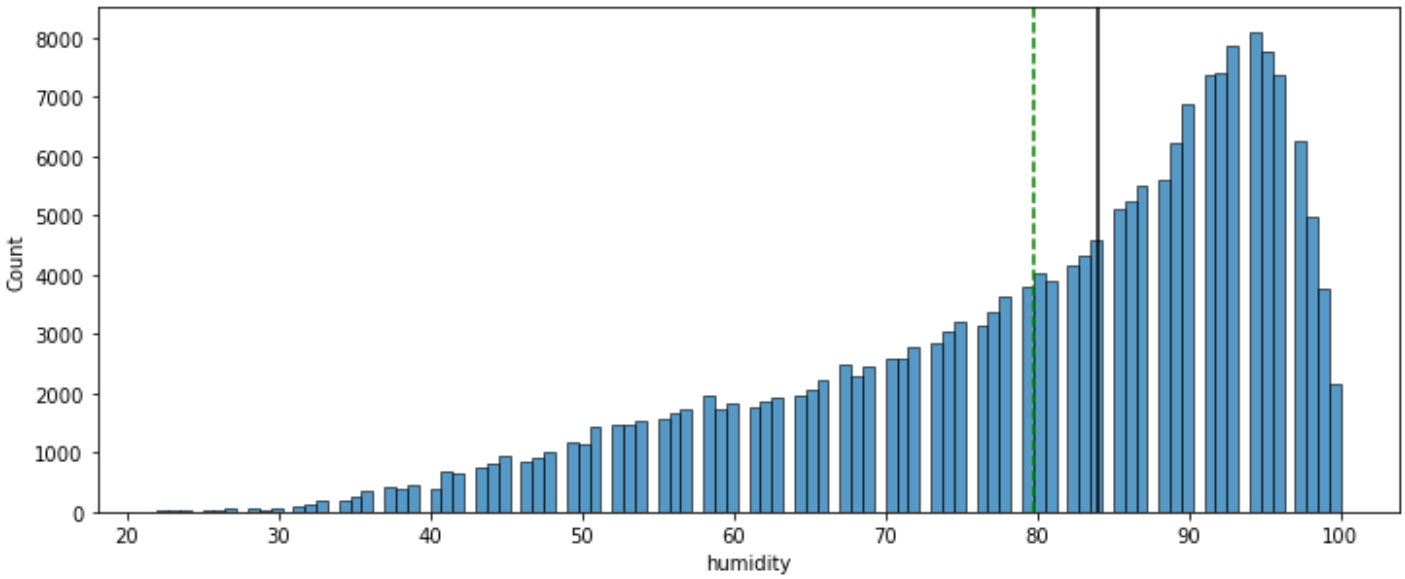


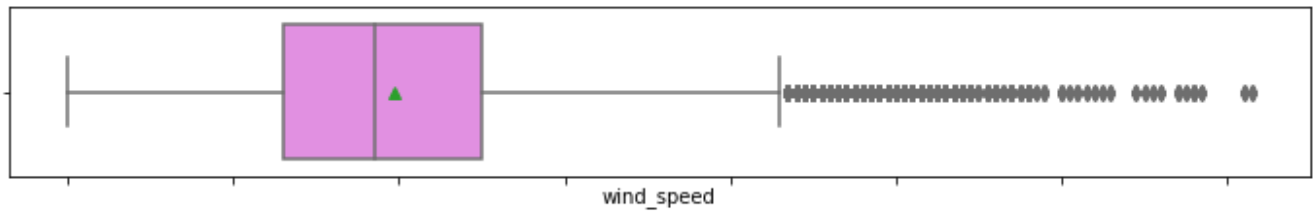


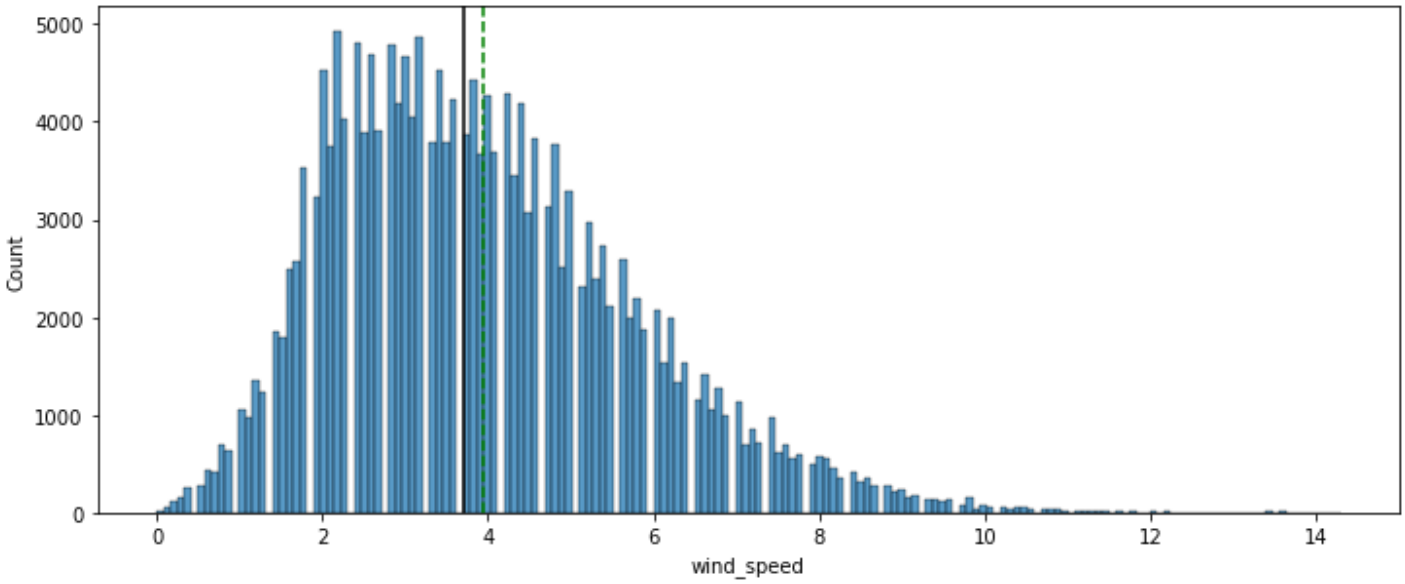


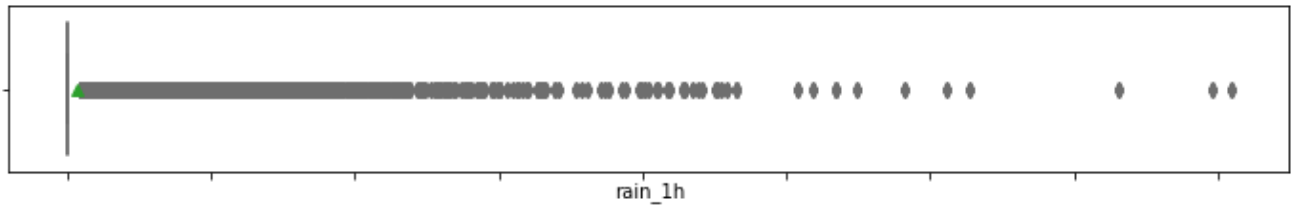


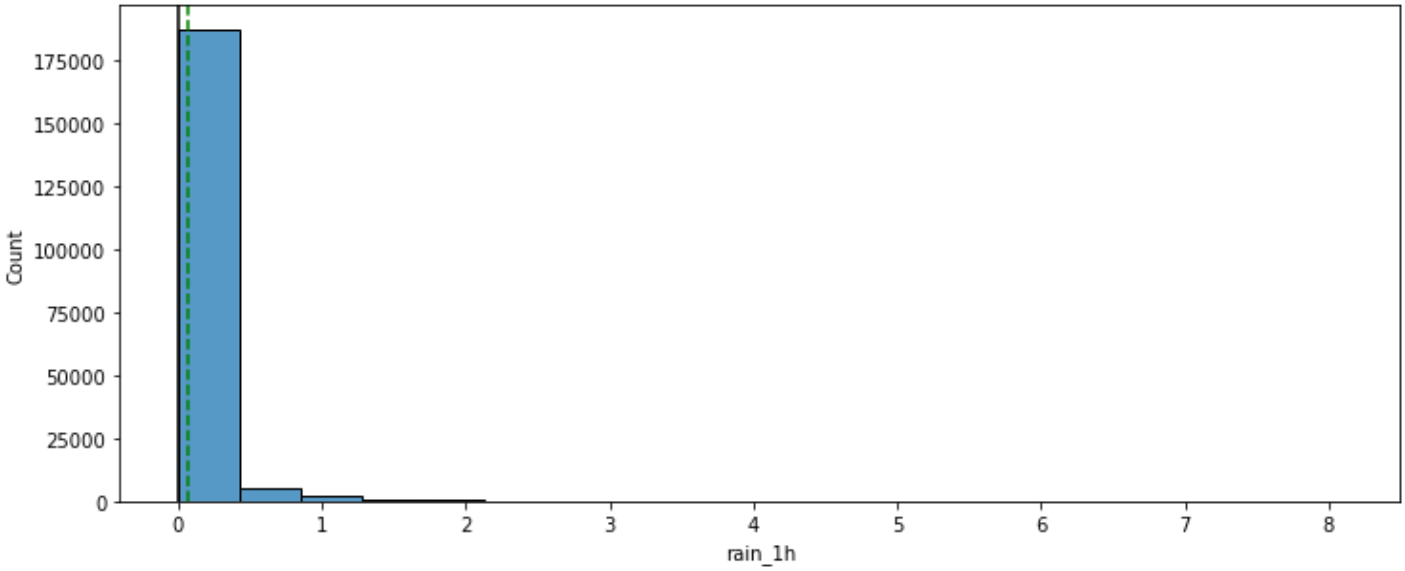


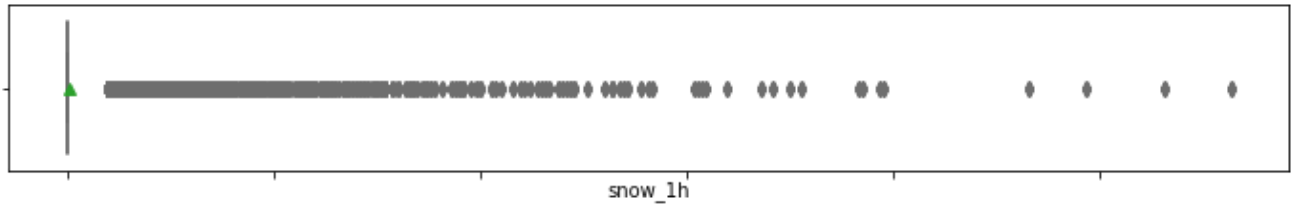


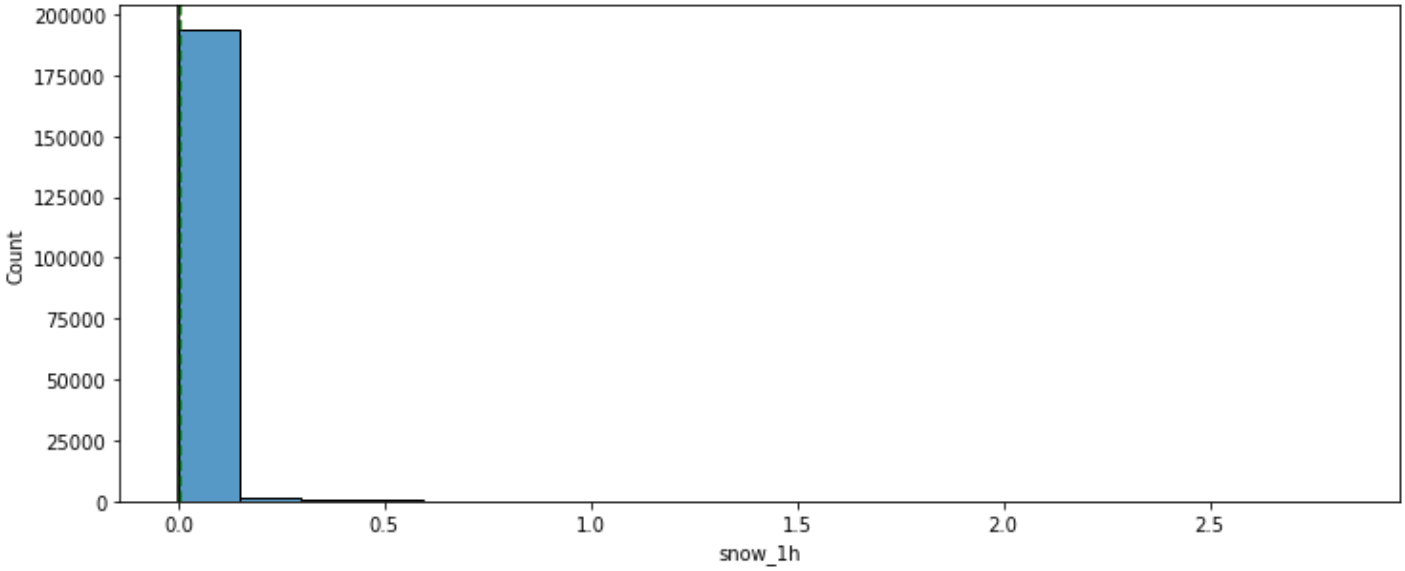


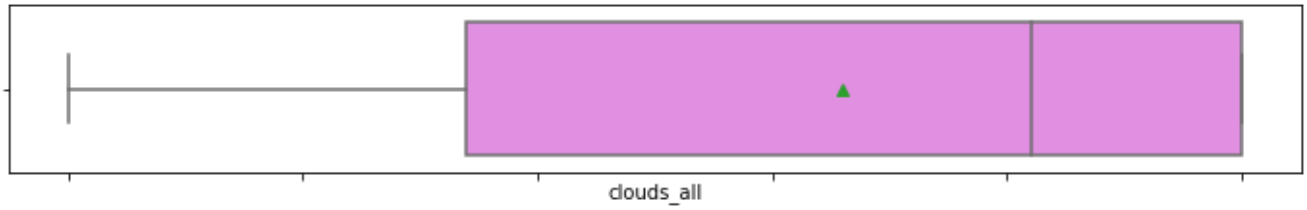


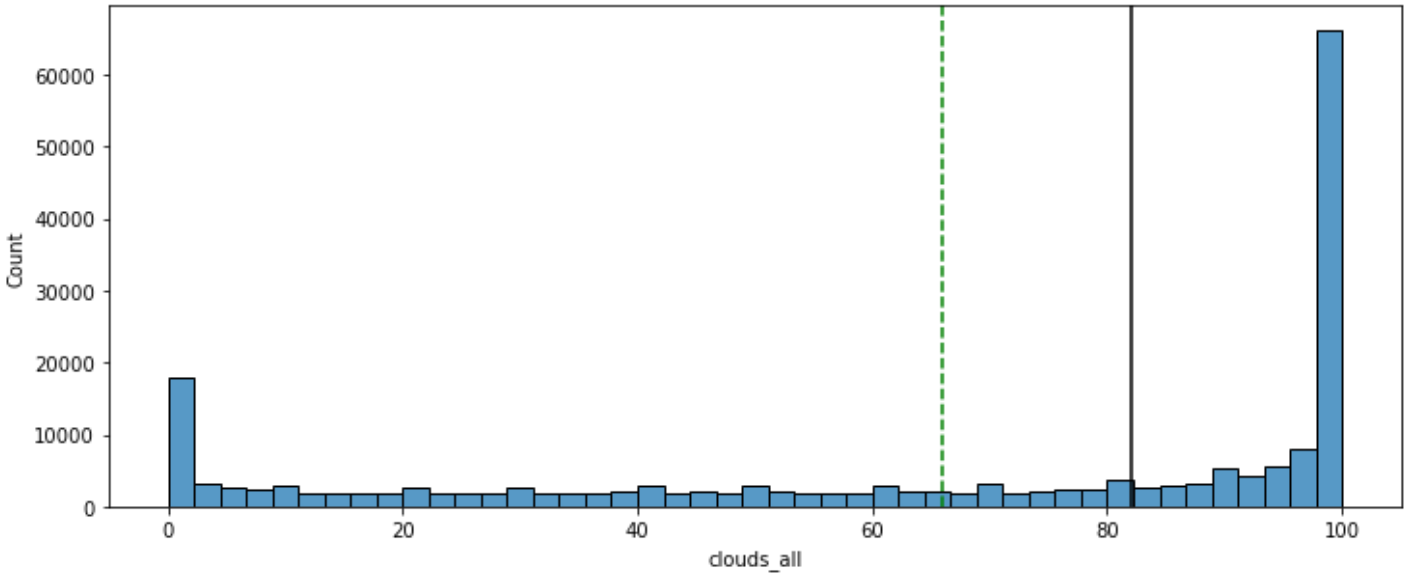


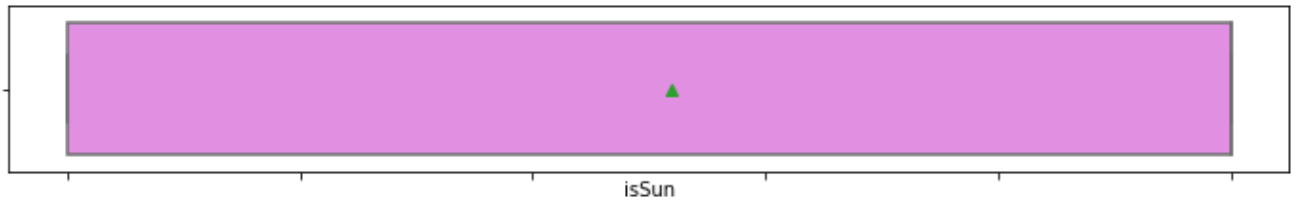


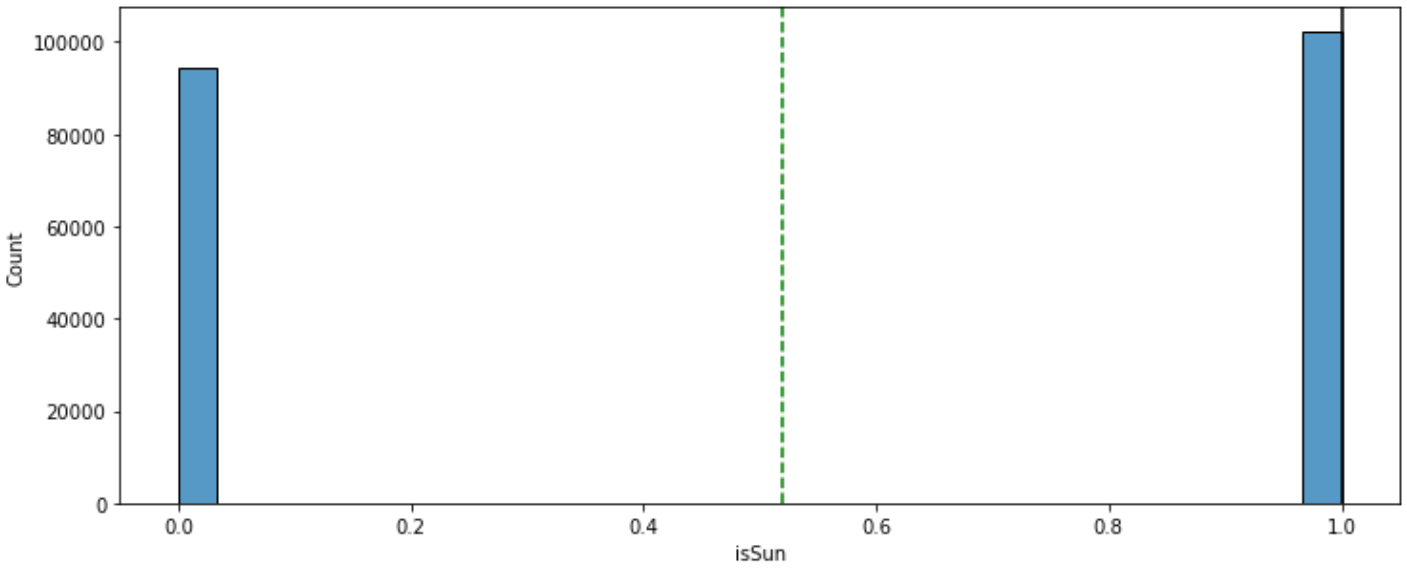


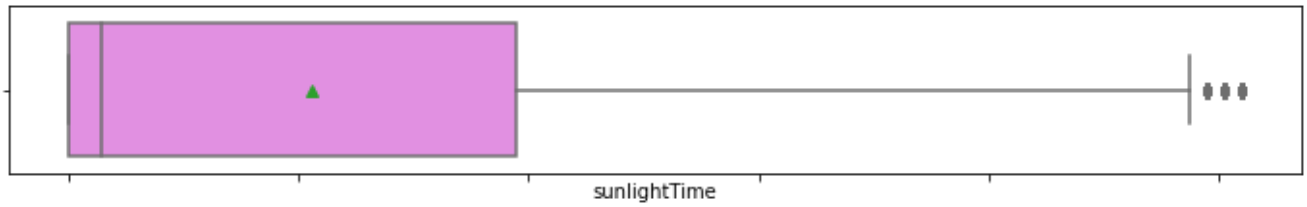


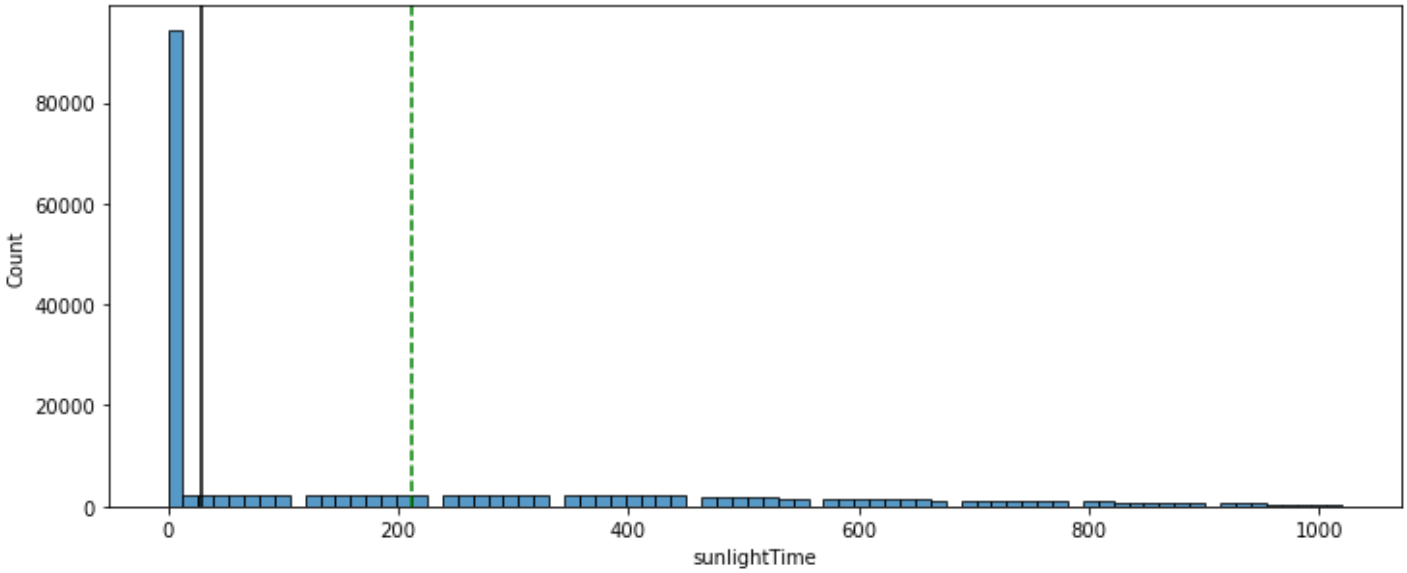


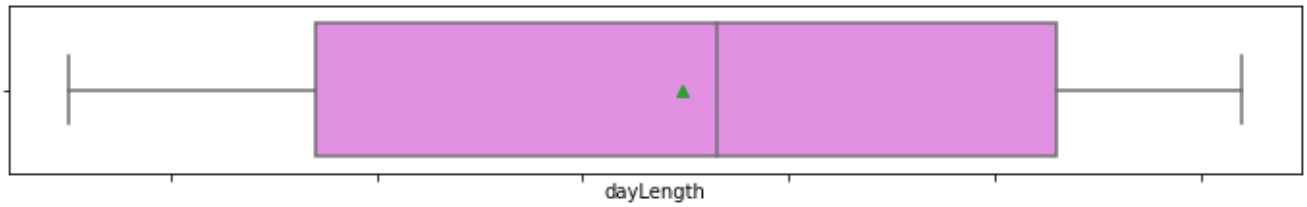


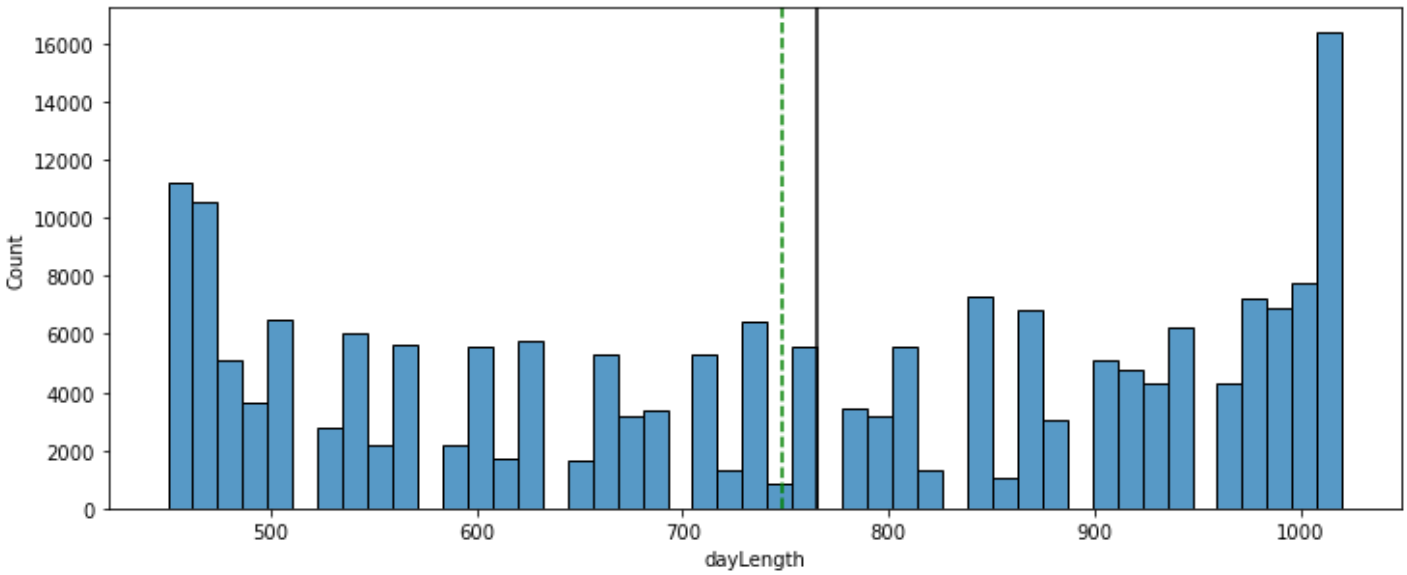


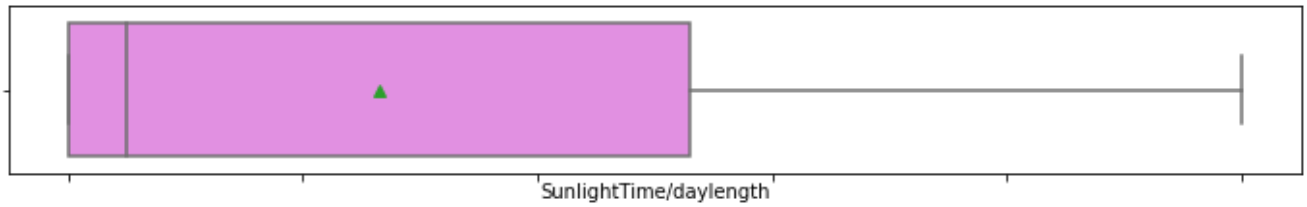


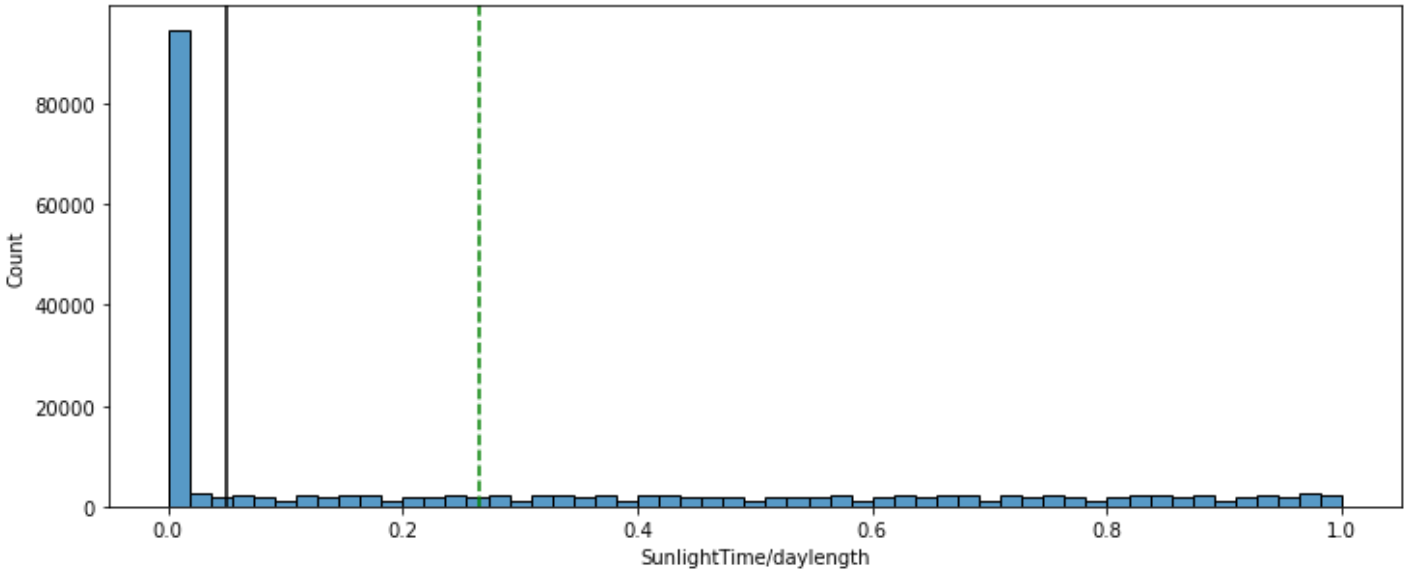


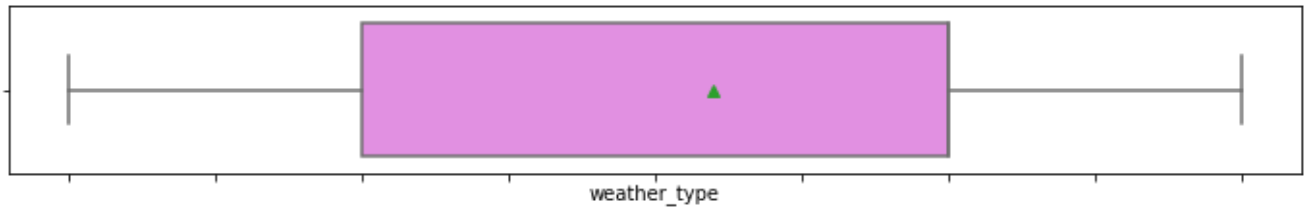


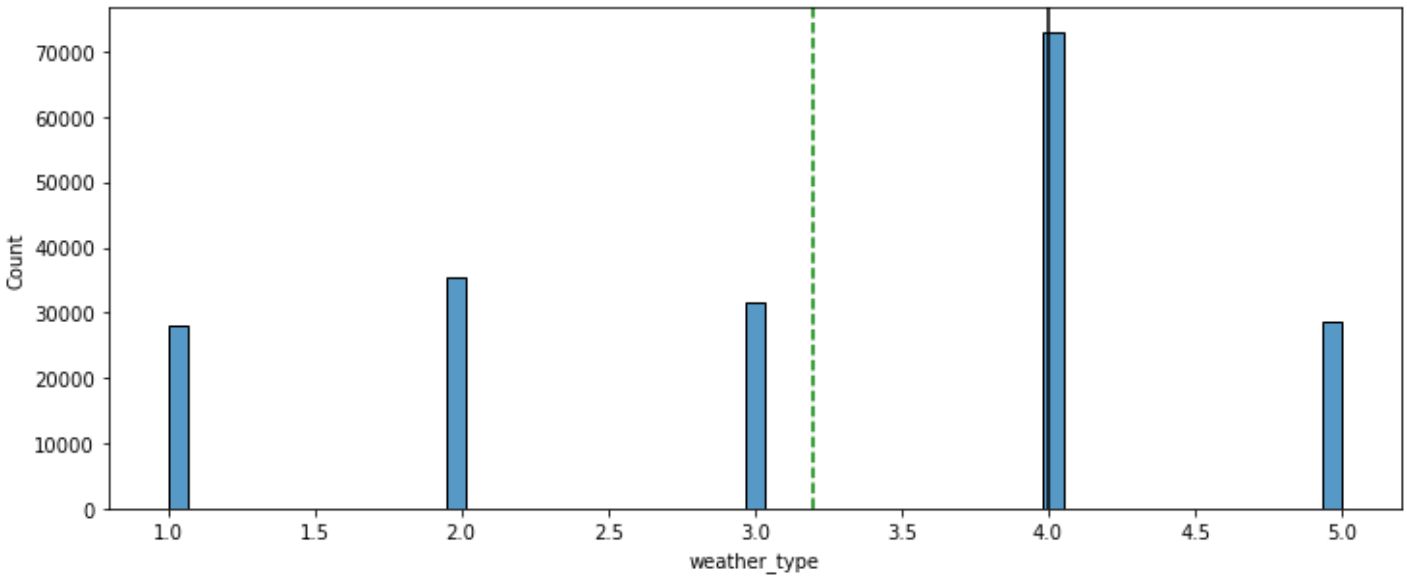


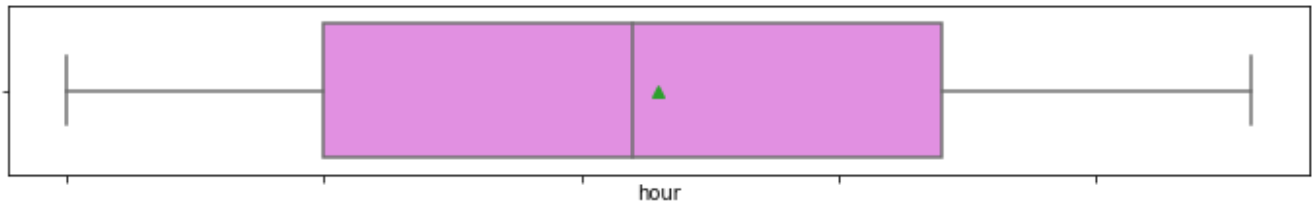


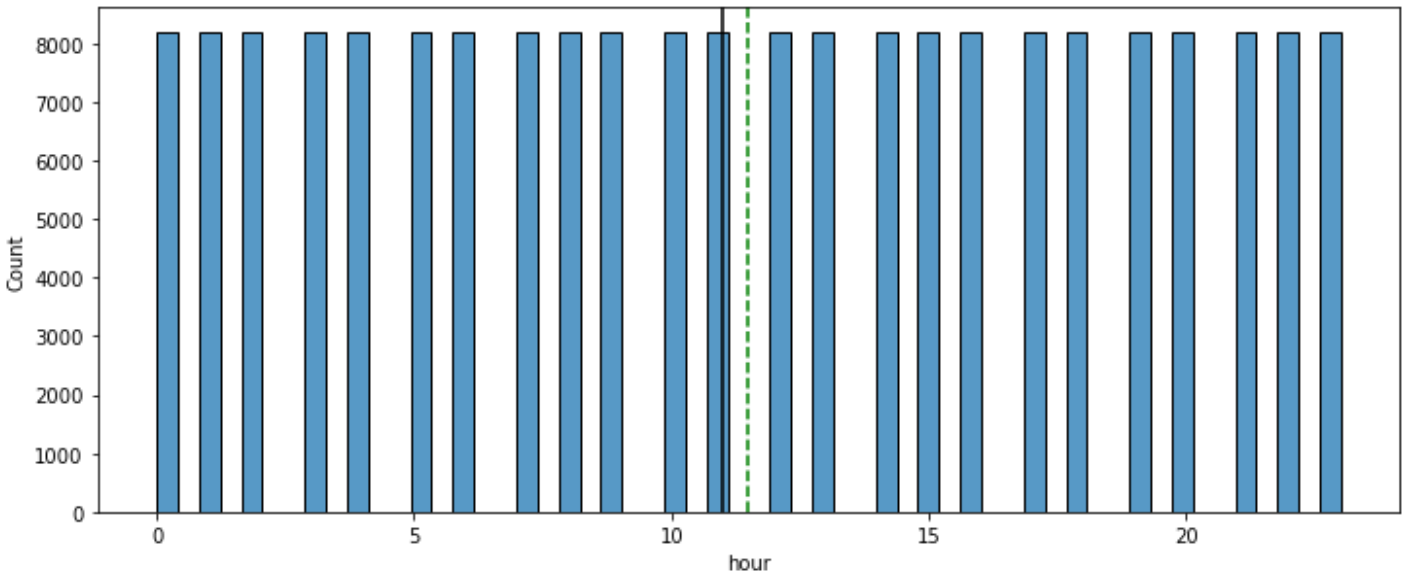


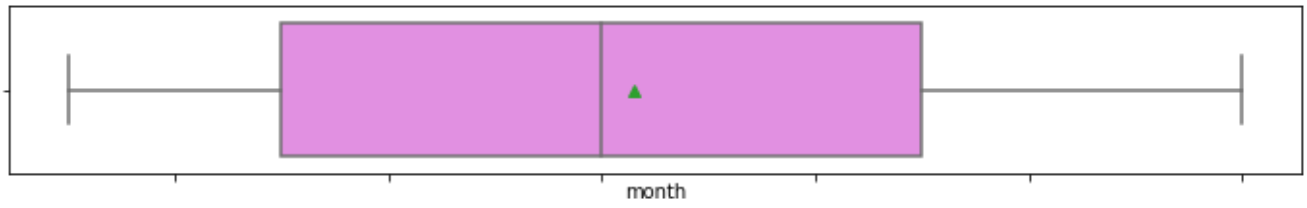


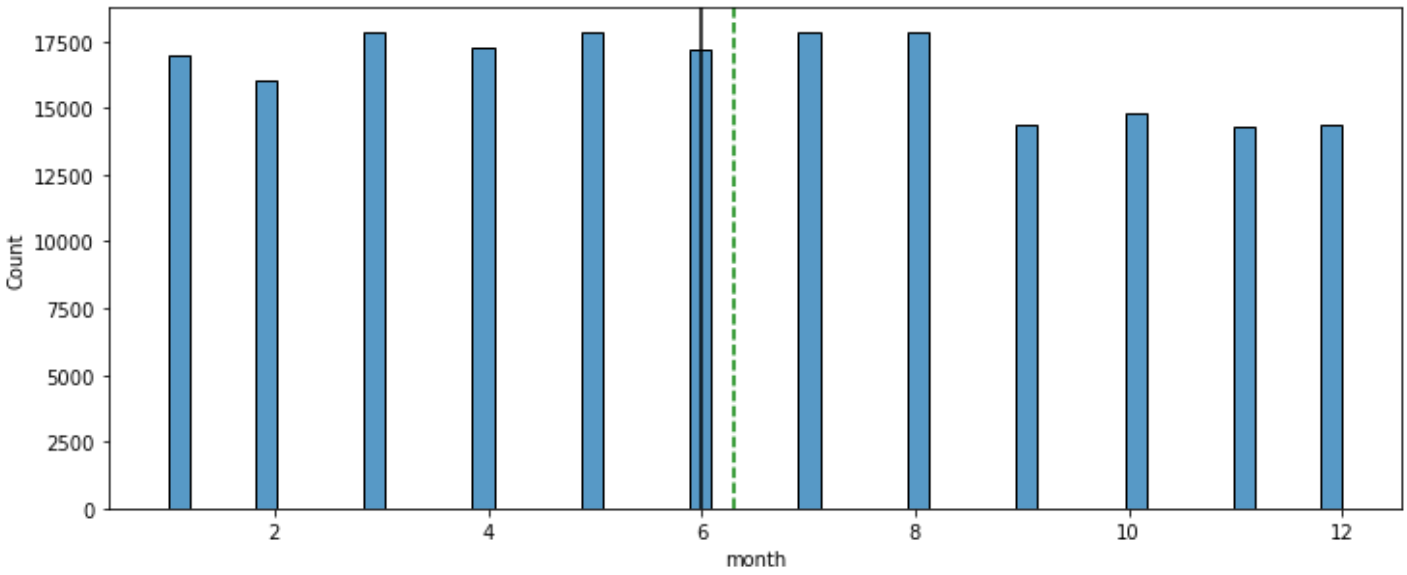


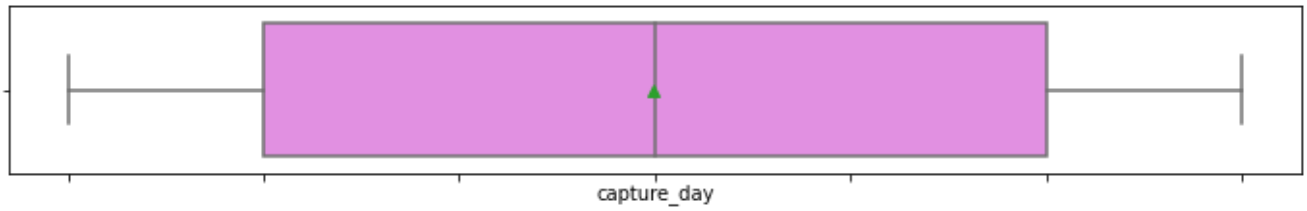


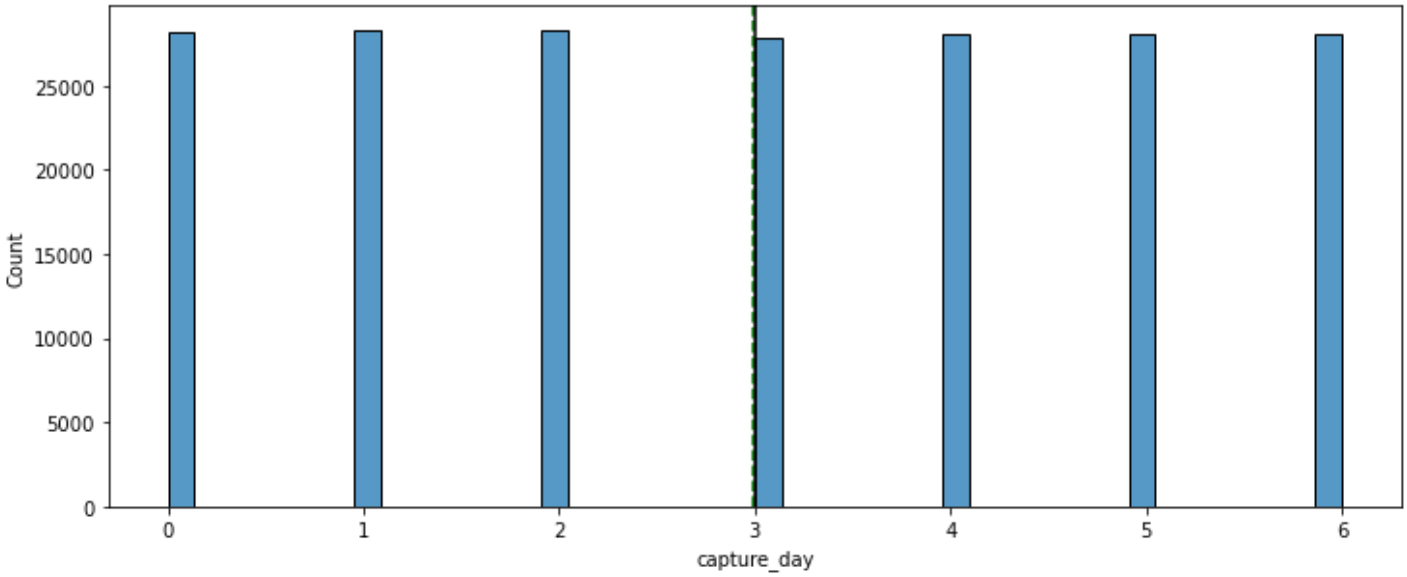


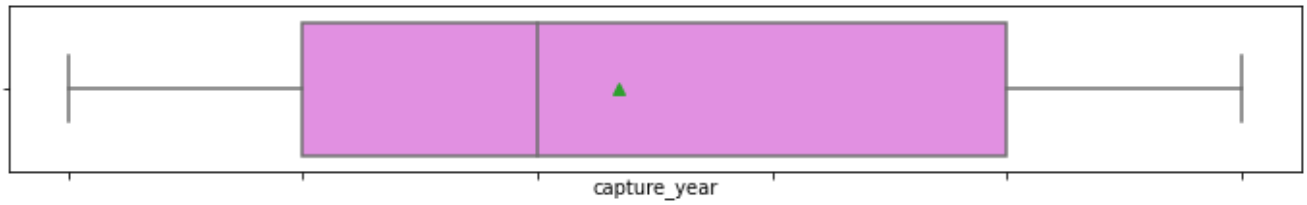


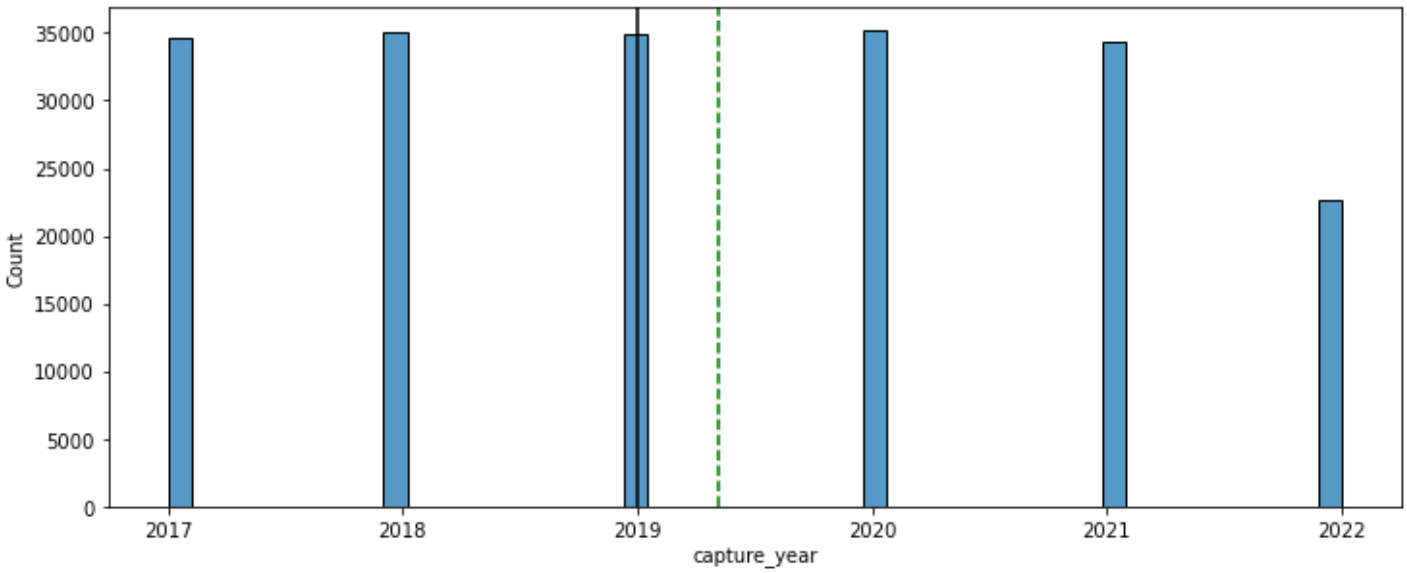


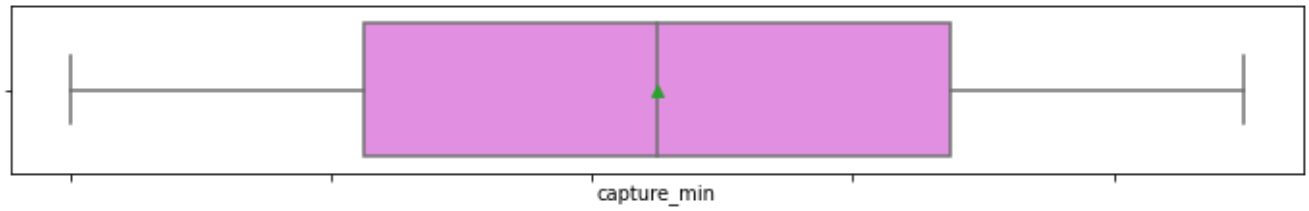


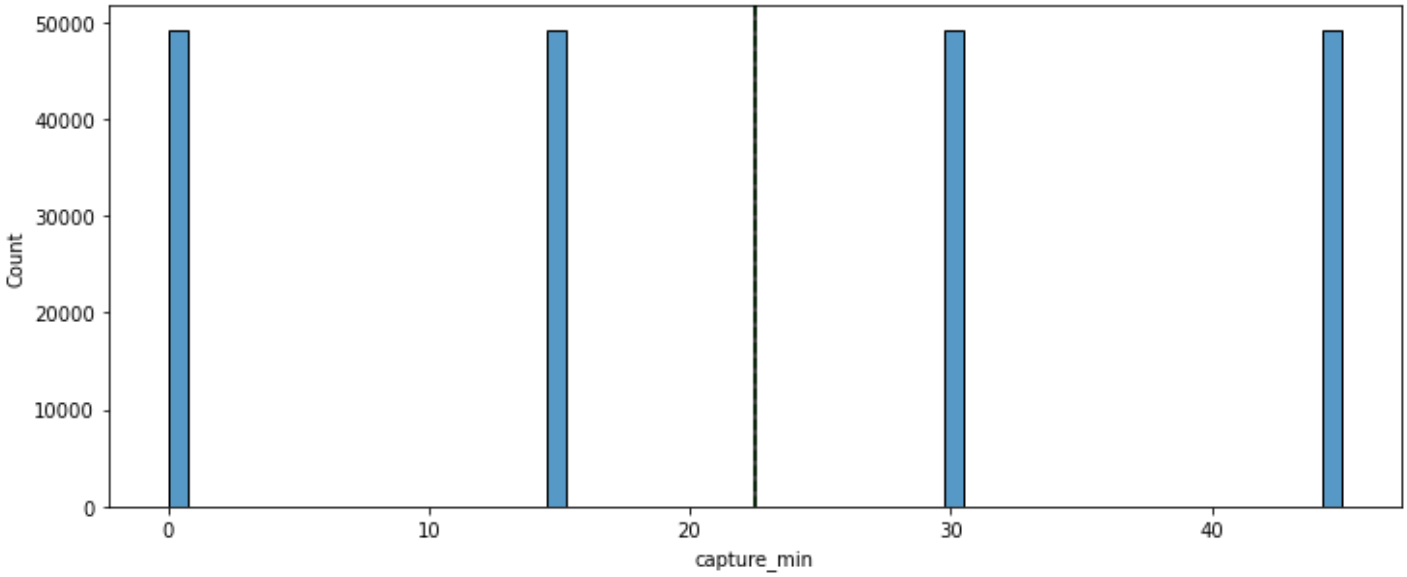








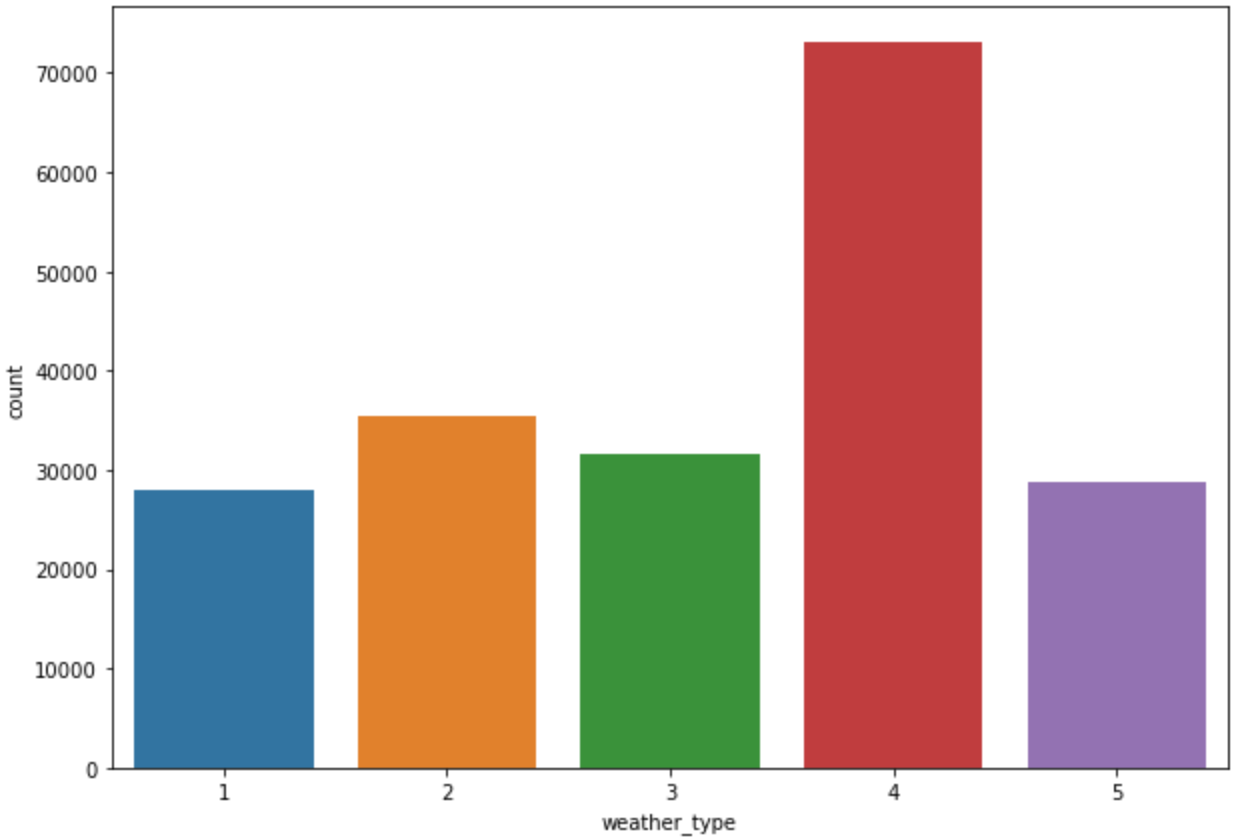


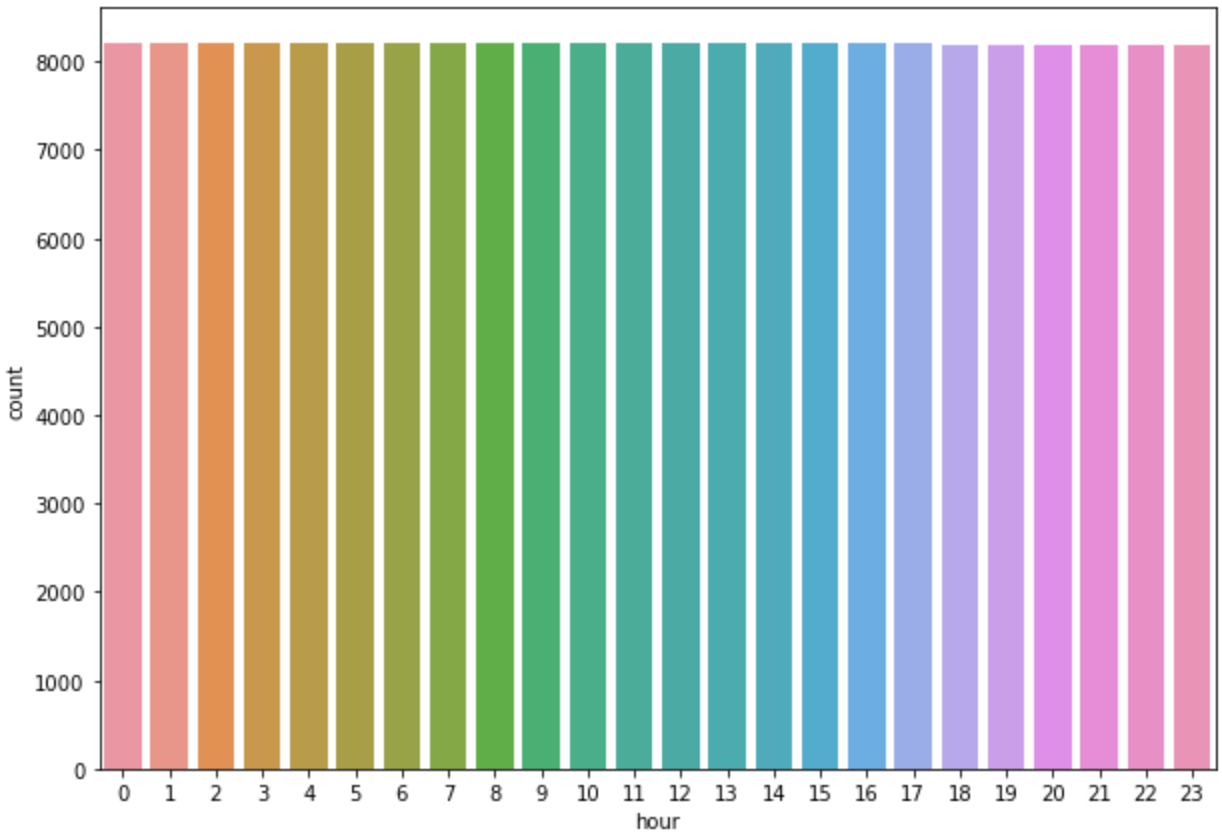


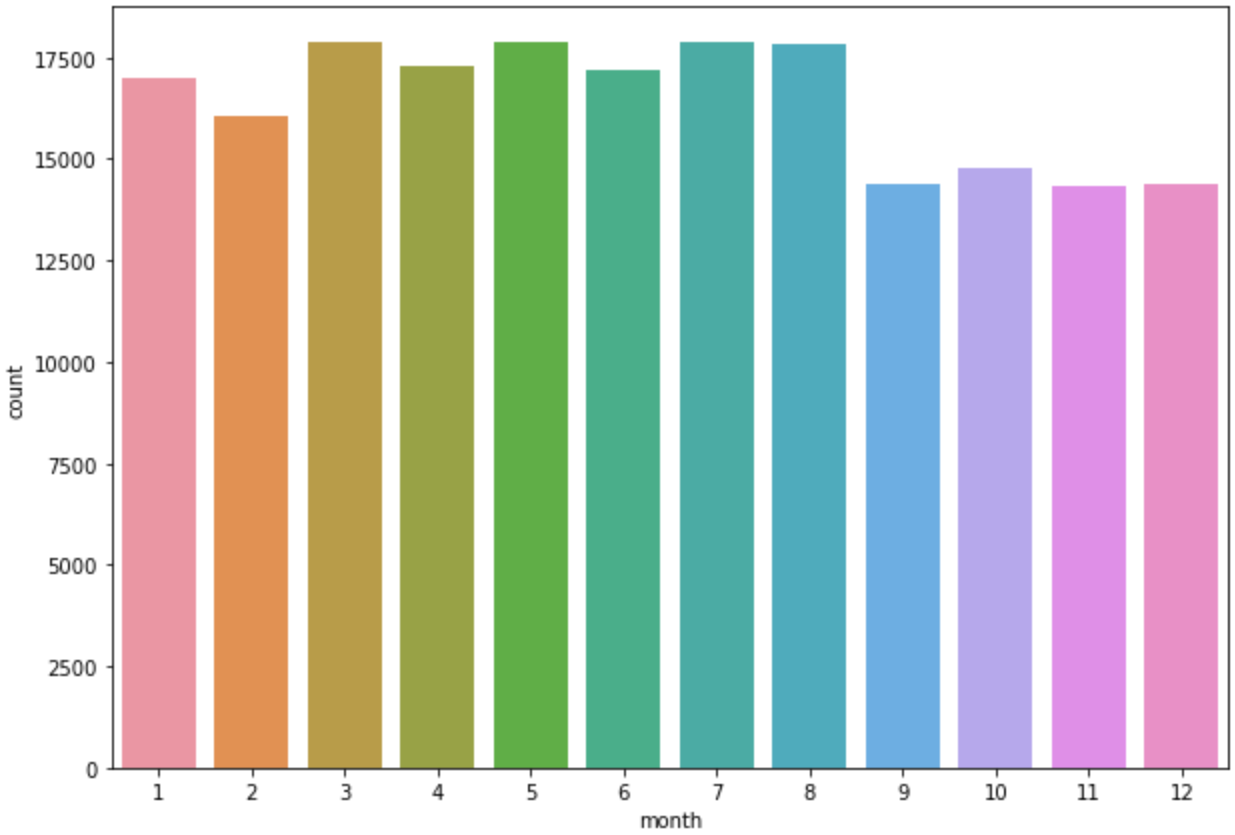
In [12]:
```
data.columns
```

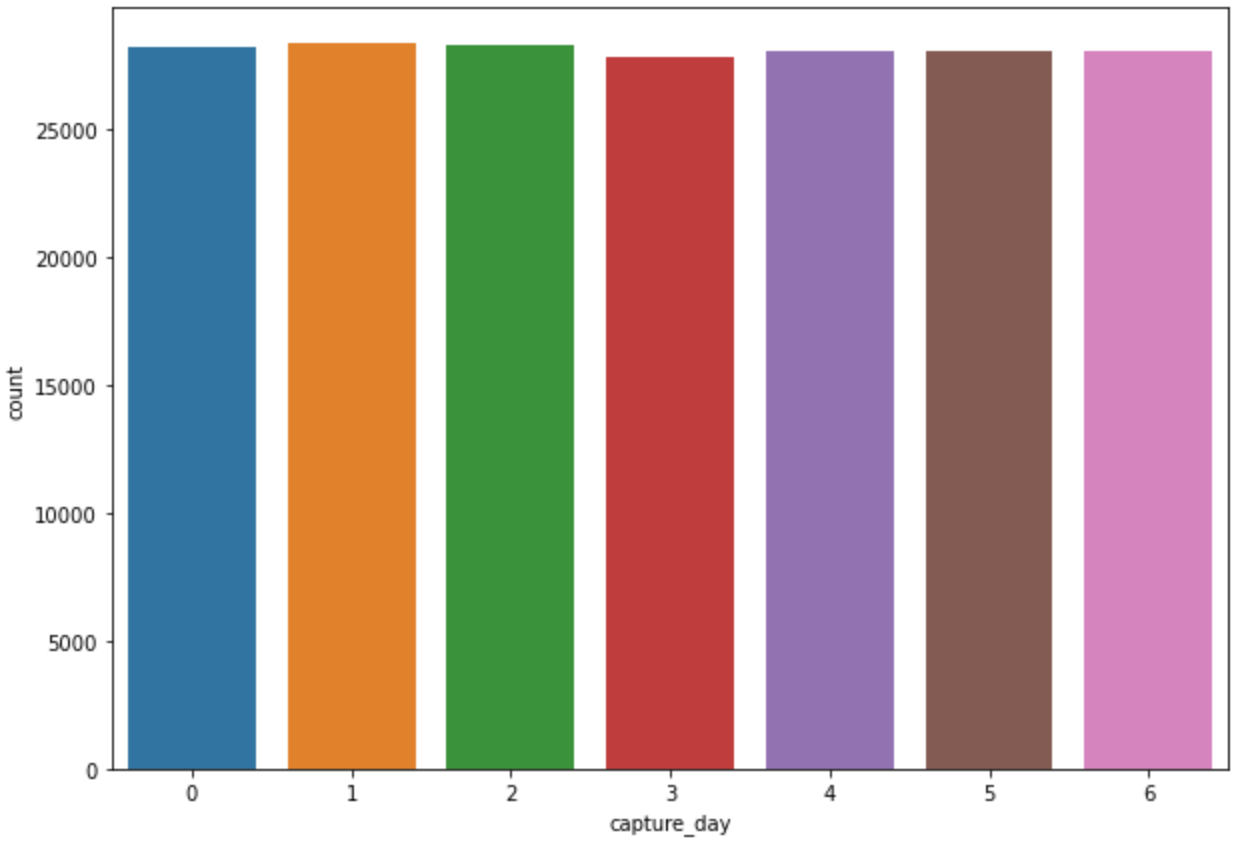
Out[12]:
```
Index(['Energy delta[Wh]', 'GHI', 'temp', 'pressure', 'humidity', 'wind_speed',
       'rain_1h', 'snow_1h', 'clouds_all', 'isSun', 'sunlightTime',
       'dayLength', 'SunlightTime/daylength', 'weather_type', 'hour', 'month',
       'capture_day', 'capture_year', 'capture_min'],
      dtype='object')
```

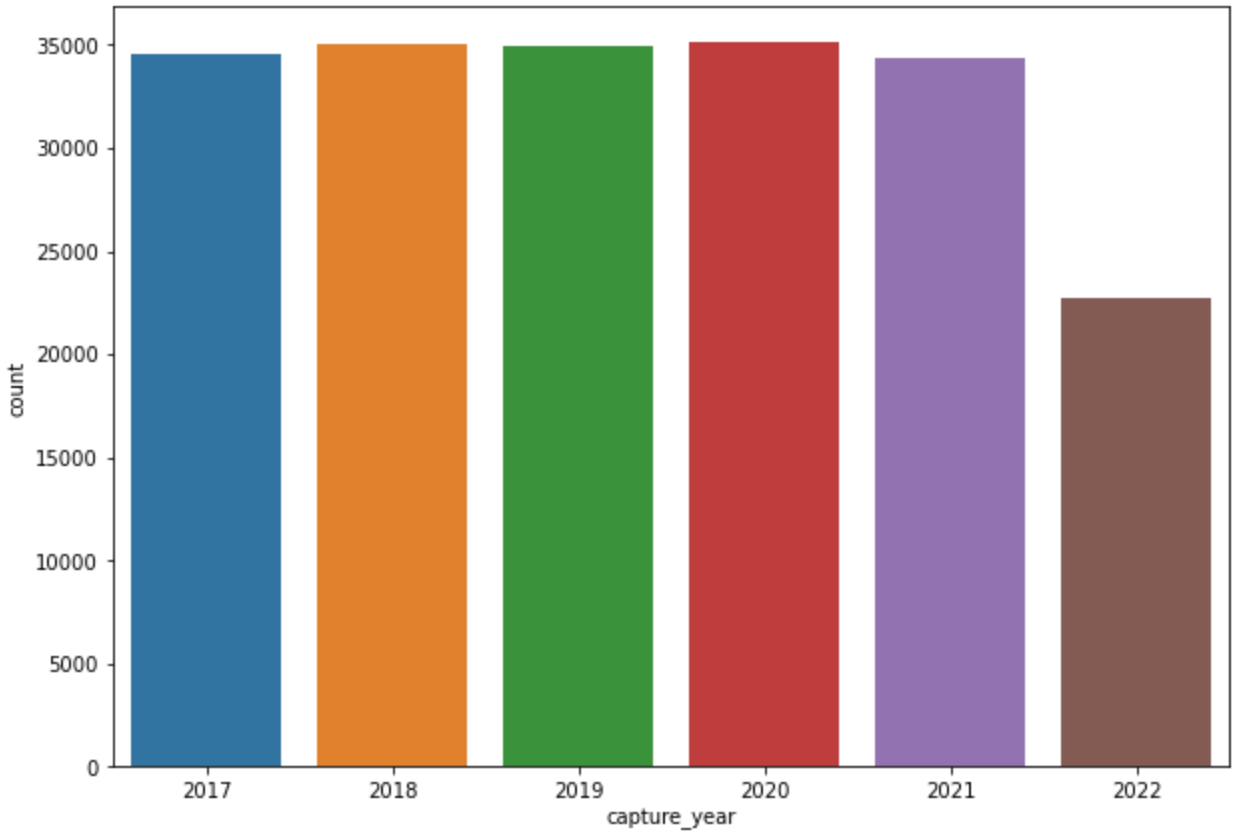
In [13]:
```
count_col=['weather_type', 'hour', 'month',
       'capture_day', 'capture_year', 'capture_min']
for i in count_col:
    plt.figure(figsize=(10,7));
    sns.countplot(data=data, x=i)
    plt.show()
```

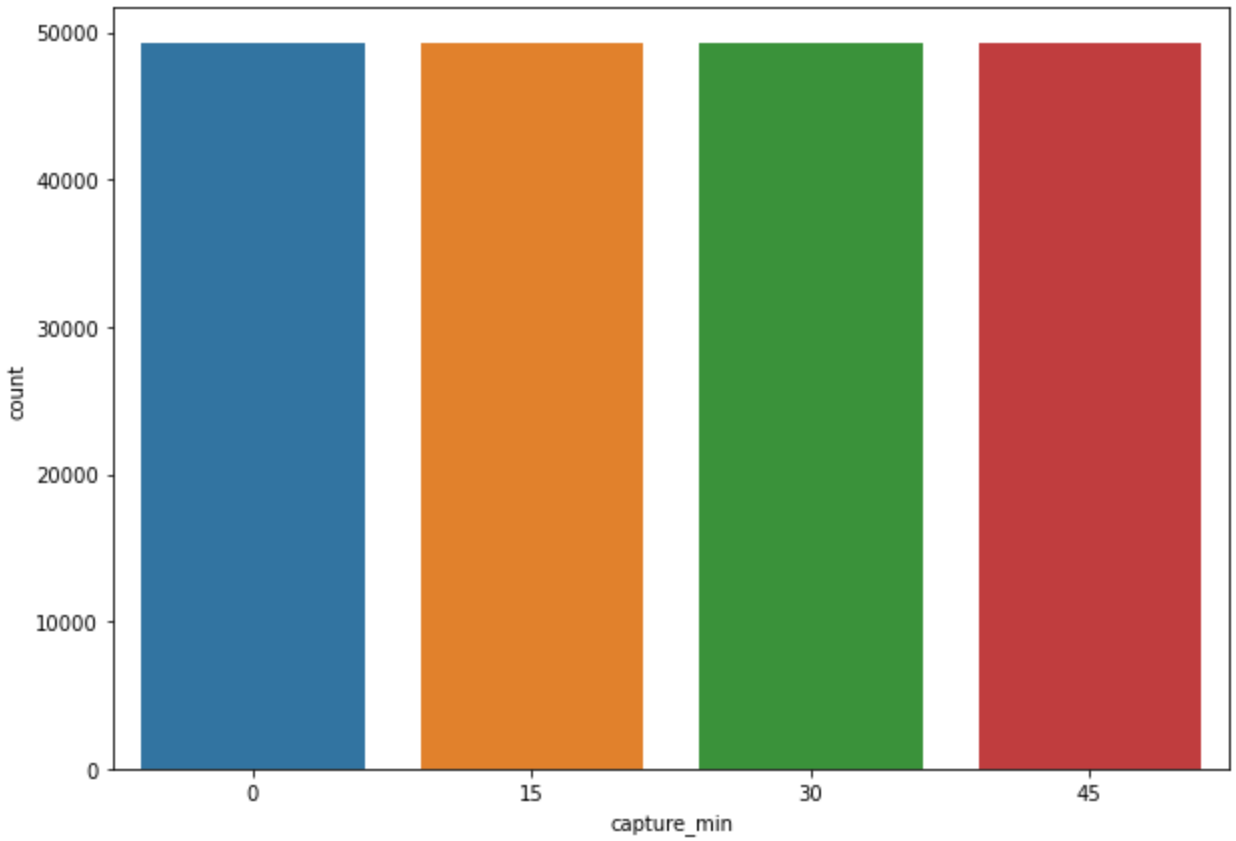






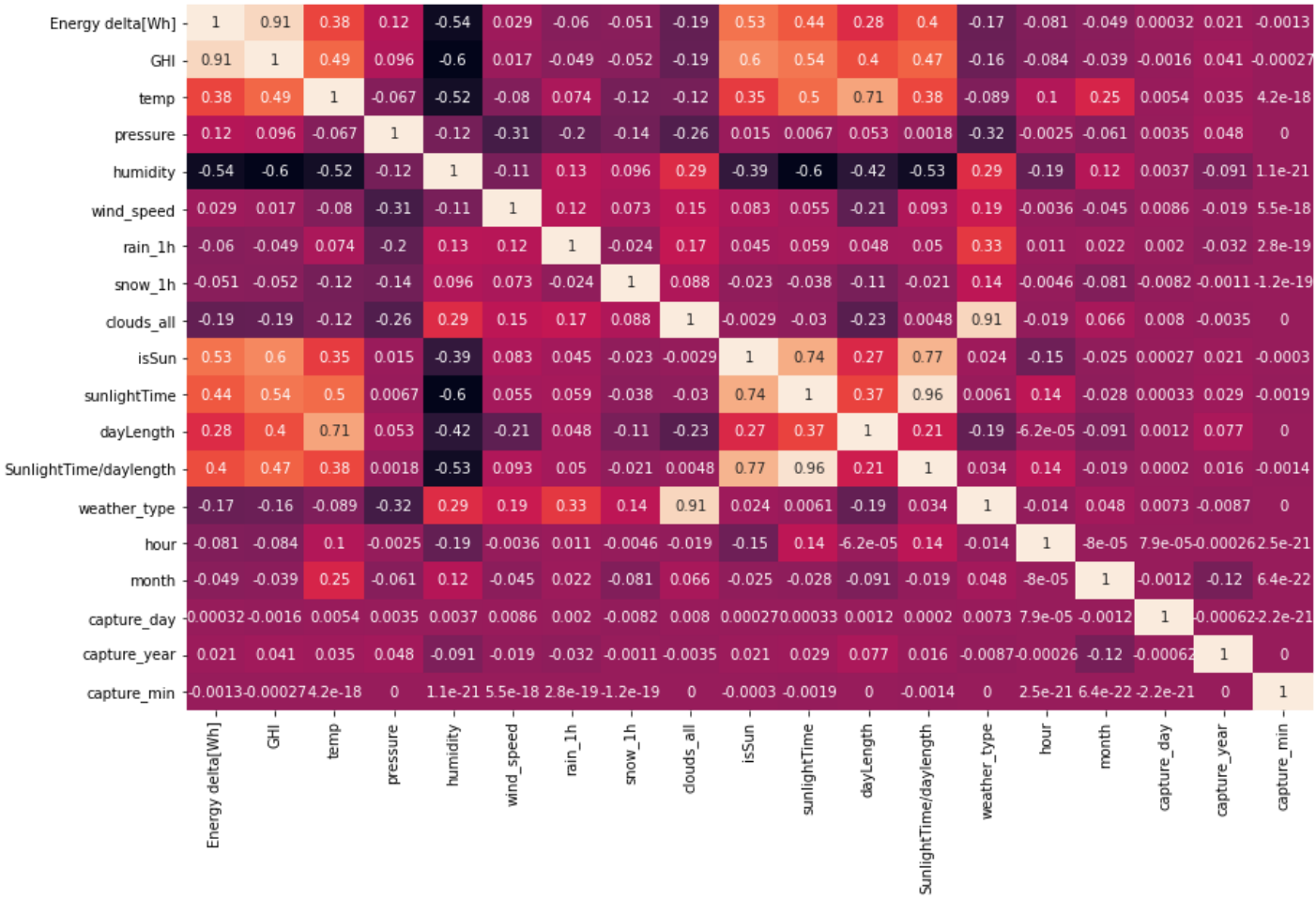






Bivariate Analysis

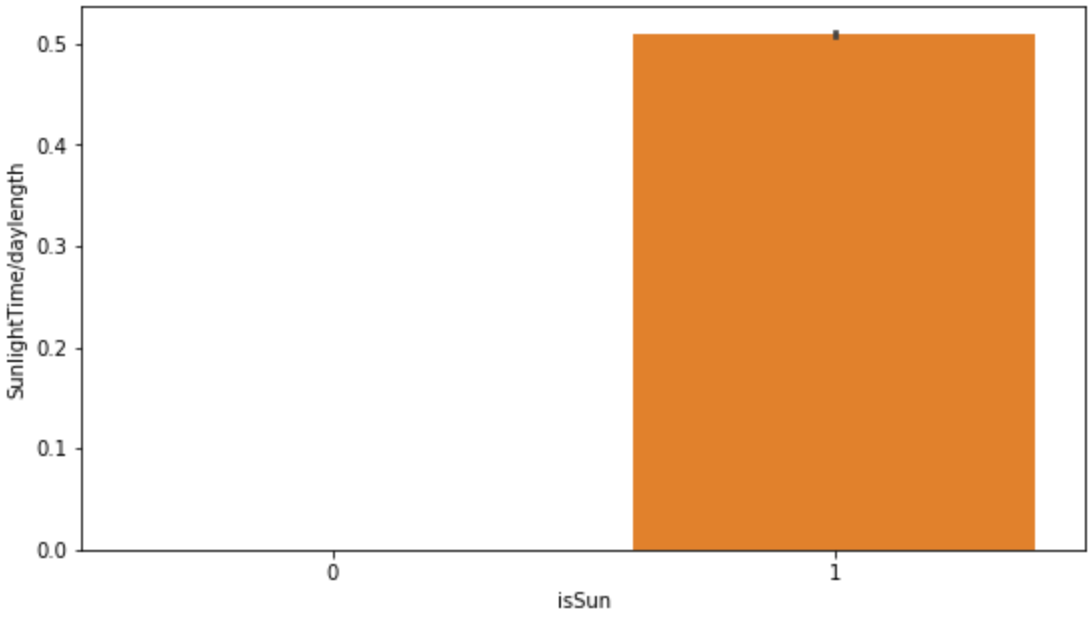
In [14]:
```
plt.figure(figsize=(14,9));
sns.heatmap(data=data.corr(), annot=True, cbar=False);
```



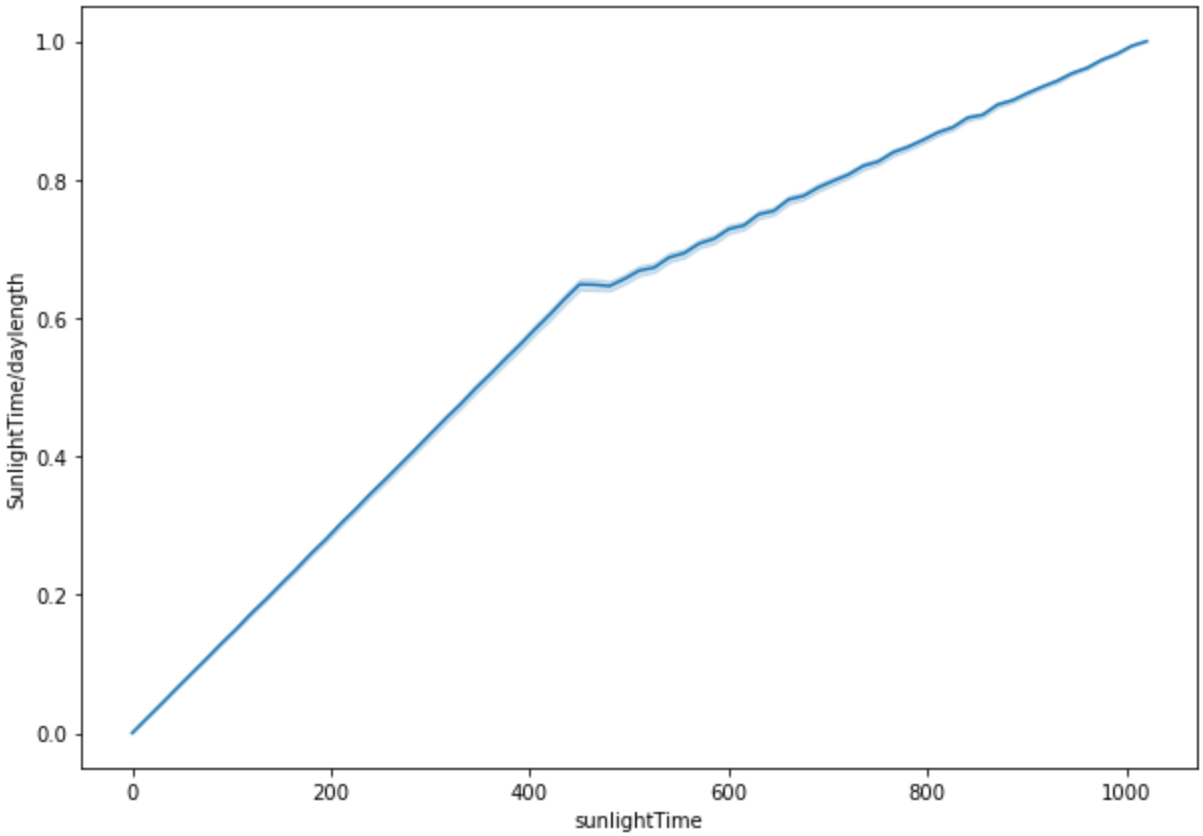
Observations

- Variables with a correlaion larger than 0.68 (positive or negative) are:
  1. SunlightTime/daylength and isSun
  2. SunlightTime/daylength and SunlightTime
  3. clouds_all and weather_type
  4. temp and dayLength
  5. Energy Delta and GHI

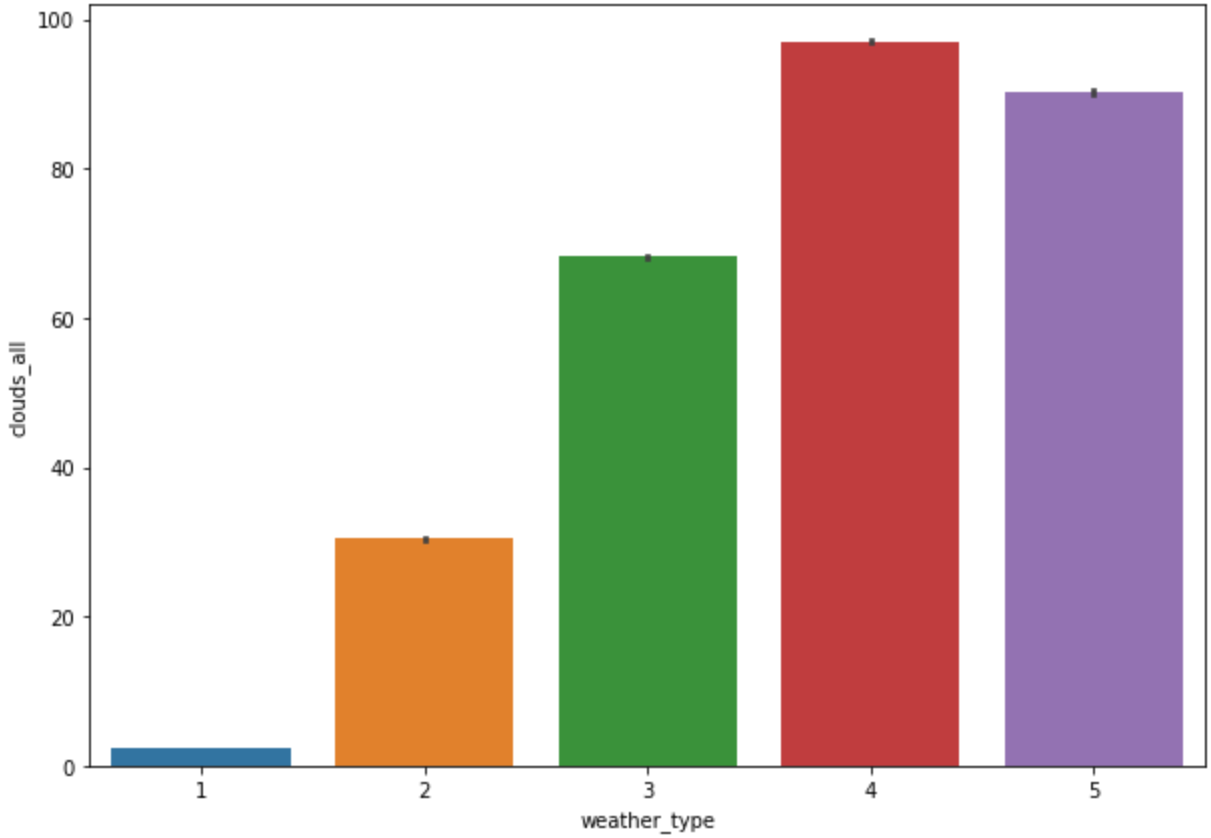
In [15]:
```
plt.figure(figsize=(9,5));
sns.barplot(data=data, y="SunlightTime/daylength", x="isSun");
```



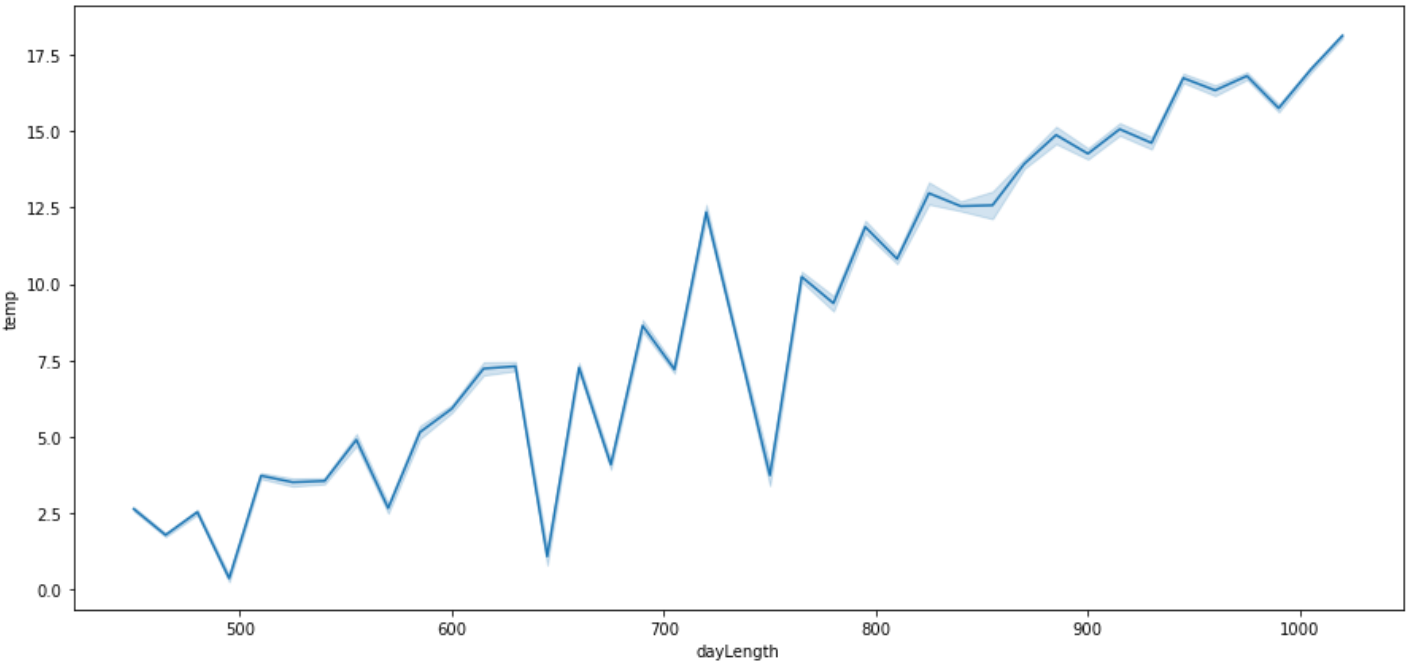
In [16]:
```
plt.figure(figsize=(10,7));
sns.lineplot(data=data, y="SunlightTime/daylength", x="sunlightTime");
```



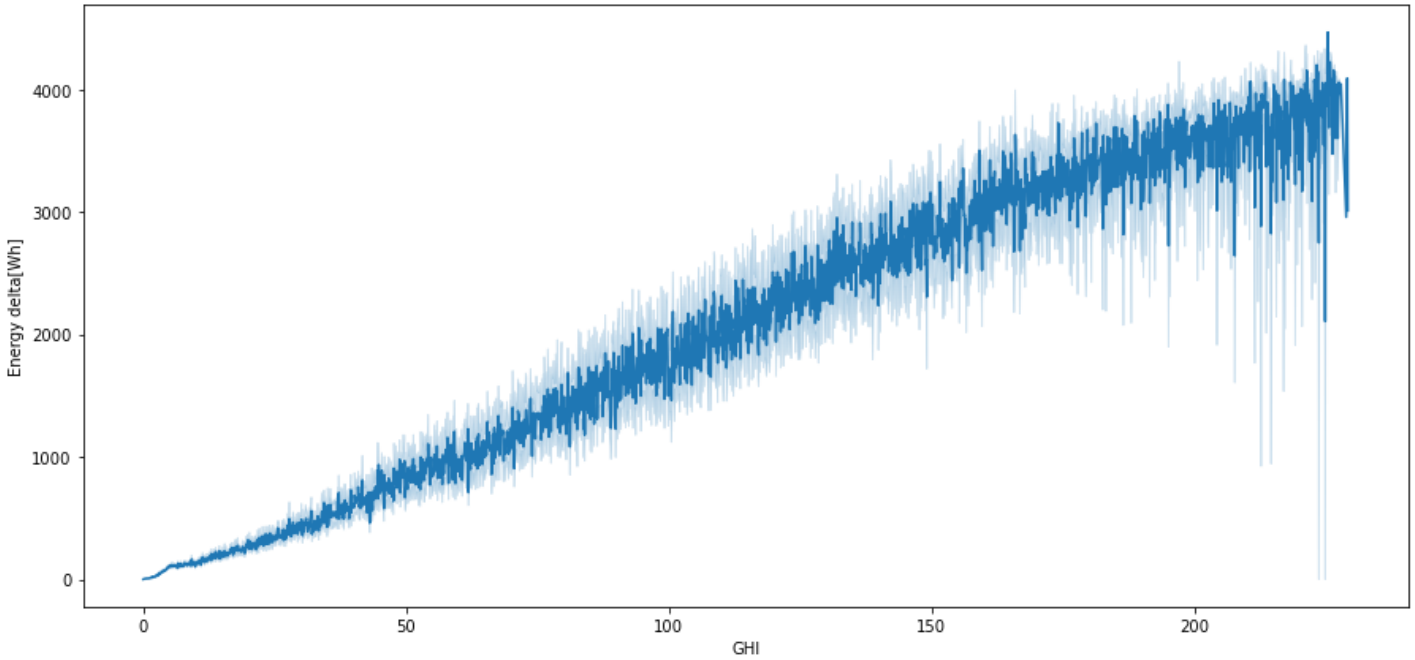
In [17]:
```
plt.figure(figsize=(10,7));
sns.barplot(data=data, y="clouds_all", x="weather_type");
```



In [18]:
```
plt.figure(figsize=(15,7));
sns.lineplot(data=data, y="temp", x="dayLength");
```



In [19]:
```
plt.figure(figsize=(15,7));
sns.lineplot(data=data, y="Energy delta[Wh]", x="GHI");
```



Data Processing

Split the data

In [20]:
```
# independent variables
X = data.drop(["Energy delta[Wh]"], axis=1)
# dependent variable
y = data[["Energy delta[Wh]"]]
```

Linear model for:

1. Weather type 1

In [21]:
```
#Grouping all the data for weather type 1
w1_x=X.loc[data["weather_type"]==1]
w1_y=y.loc[data["weather_type"]==1]
```

In [22]:
```
# Building the linear model
w1_x=sm.add_constant(w1_x)

#Splitting the data into training and testing by the ratio 7:3
w1_x_train, w1_x_test, w1_y_train, w1_y_test = train_test_split(
    w1_x, w1_y, test_size=0.30, random_state=1
)
```

In [23]:
```
#Fitting the model
w1_olsmodel=sm.OLS(w1_y_train, w1_x_train).fit()
print(w1_olsmodel.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:       Energy delta[Wh]   R-squared:                       0.904
Model:                            OLS   Adj. R-squared:                  0.904
Method:                 Least Squares   F-statistic:                 1.231e+04
Date:                Sat, 01 Jul 2023   Prob (F-statistic):               0.00
Time:                        11:57:33   Log-Likelihood:            -1.4580e+05
No. Observations:               19583   AIC:                         2.916e+05
Df Residuals:                   19567   BIC:                         2.918e+05
Df Model:                          15
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
```

```
--------------------------------------------------------------------------------
GHI                         21.2391      0.077    274.693      0.000      21.088      21.391
temp                        -2.2012      0.683     -3.223      0.001      -3.540      -0.863
pressure                     1.2729      0.415      3.071      0.002       0.460       2.085
humidity                    -1.6468      0.294     -5.592      0.000      -2.224      -1.070
wind_speed                  -1.7762      2.482     -0.716      0.474      -6.641       3.088
rain_1h                 -4.652e-10   3.03e-10     -1.537      0.124   -1.06e-09    1.28e-10
snow_1h                 -1.084e-10   7.15e-11     -1.516      0.130   -2.49e-10    3.18e-11
clouds_all                  -1.8276      0.981     -1.862      0.063      -3.751       0.096
isSun                     -172.0519     12.119    -14.197      0.000    -195.806    -148.297
sunlightTime                -3.4070      0.058    -59.193      0.000      -3.520      -3.294
dayLength                   -0.5781      0.029    -19.973      0.000      -0.635      -0.521
SunlightTime/daylength    2798.5320     51.437     54.407      0.000    2697.711    2899.354
weather_type              5968.5416   3866.427      1.544      0.123   -1609.985    1.35e+04
hour                         1.9711      0.471      4.187      0.000       1.048       2.894
month                       -8.0761      1.375     -5.875      0.000     -10.770      -5.382
capture_day                  2.3780      1.443      1.648      0.099      -0.450       5.206
capture_year                -3.2913      1.937     -1.699      0.089      -7.089       0.506
capture_min                 -0.2730      0.177     -1.543      0.123      -0.620       0.074
==============================================================================
Omnibus:                     2826.791   Durbin-Watson:                   1.977
Prob(Omnibus):                  0.000   Jarque-Bera (JB):            32481.529
Skew:                          -0.301   Prob(JB):                         0.00
Kurtosis:                       9.281   Cond. No.                     2.00e+19
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specifie
d.
[2] The smallest eigenvalue is 2.85e-28. This might indicate that there are
strong multicollinearity problems or that the design matrix is singular.
```

Multicolinearity

In [24]:
```python
# we will define a function to check VIF
def checking_vif(predictors):
    vif = pd.DataFrame()
    vif["feature"] = predictors.columns

    # calculating VIF for each feature
    vif["VIF"] = [
        variance_inflation_factor(predictors.values, i)
        for i in range(len(predictors.columns))
    ]
    return vif
```

In [25]:
```python
#Calculating the VIF of each column
checking_vif(w1_x_train)
```

Out[25]:

	feature	VIF
0	GHI	2.824437e+00
1	temp	4.331760e+00
2	pressure	1.356921e+00
3	humidity	3.319761e+00
4	wind_speed	1.341271e+00
5	rain_1h	NaN
6	snow_1h	NaN
7	clouds_all	1.082874e+00
8	isSun	4.183052e+00
	ghtTime	2.922950e+01

	feature	VIF
10	dayLength	2.626387e+00
11	SunlightTime/daylength	3.167072e+01
12	weather_type	1.704324e+06
13	hour	1.380107e+00
14	month	1.680249e+00
15	capture_day	1.006338e+00
16	capture_year	1.077149e+00
17	capture_min	1.000925e+00

In [26]:
```
#Analysis of rain1h and snow1h
print(w1_x_train.rain_1h.value_counts(normalize=True))
print(w1_x_train.snow_1h.value_counts(normalize=True))
```

```
0.0    1.0
Name: rain_1h, dtype: float64
0.0    1.0
Name: snow_1h, dtype: float64
```

In [27]:
```
#Dropping rain and snow columns as they only have 0 values for this subset
w1_x_train.drop(["rain_1h", "snow_1h"], axis=1, inplace=True)
w1_x_train
```

Out[27]:

	GHI	temp	pressure	humidity	wind_speed	clouds_all	isSun	sunlightTime	dayLength	SunlightTime/day
2172	5.7	-0.5	1021	74	3.2	4	1	510	525	
69511	0.0	2.7	1027	97	3.1	8	0	0	450	
61812	0.0	16.3	1021	72	5.6	0	0	0	660	
19683	0.0	14.1	1013	98	0.5	3	0	0	945	
86679	0.0	12.4	1023	92	2.0	0	0	0	1020	
...	...	...	...	...	...	...	...	...	...	
74287	0.0	4.3	1017	84	2.6	0	0	0	600	
114169	73.9	3.0	1027	86	2.3	0	1	135	810	
45196	0.0	10.4	1020	70	2.3	0	0	0	870	
79848	0.0	12.6	1029	52	3.4	0	0	0	840	
2111	0.0	-3.8	1024	87	2.4	0	0	0	525	

19583 rows × 16 columns

In [28]:
```
#Checking the new VIF after dropping the columns
checking_vif(w1_x_train)
```

Out[28]:

	feature	VIF
0	GHI	2.824437e+00
1	temp	4.331760e+00
2	pressure	1.356921e+00
3	humidity	3.319761e+00
4	wind_speed	1.341271e+00

	feature	VIF
5	clouds_all	1.082874e+00
6	isSun	4.183052e+00
7	sunlightTime	2.922950e+01
8	dayLength	2.626387e+00
9	SunlightTime/daylength	3.167072e+01
10	weather_type	1.704324e+06
11	hour	1.380107e+00
12	month	1.680249e+00
13	capture_day	1.006338e+00
14	capture_year	1.077149e+00
15	capture_min	1.000925e+00

In [29]:
```
def treating_multicollinearity(predictors, target, high_vif_columns):
    """
    Checking the effect of dropping the columns showing high multicollinearity
    on model performance (adj. R-squared and RMSE)

    predictors: independent variables
    target: dependent variable
    high_vif_columns: columns having high VIF
    """
    # empty lists to store adj. R-squared and RMSE values
    adj_r2 = []
    rmse = []

    # build ols models by dropping one of the high VIF columns at a time
    # store the adjusted R-squared and RMSE in the lists defined previously
    for cols in high_vif_columns:
        # defining the new train set
        train = predictors.loc[:, ~predictors.columns.str.startswith(cols)]

        # create the model
        olsmodel = sm.OLS(target, train).fit()

        # adding adj. R-squared and RMSE to the lists
        adj_r2.append(olsmodel.rsquared_adj)
        rmse.append(np.sqrt(olsmodel.mse_resid))

    # creating a dataframe for the results
    temp = pd.DataFrame(
        {
            "col": high_vif_columns,
            "Adj. R-squared after_dropping col": adj_r2,
            "RMSE after dropping col": rmse,
        }
    ).sort_values(by="Adj. R-squared after_dropping col", ascending=False)
    temp.reset_index(drop=True, inplace=True)

    return temp
```

In [30]:
```
#Original Model asjusted R value 0.842
col_list=["weather_type","SunlightTime/daylength","sunlightTime"]
res = treating_multicollinearity(w1_x_train, w1_y_train, col_list)
res
```

Out[30]:

	col	Adj. R-squared after_dropping col	RMSE after dropping col
0	weather_type	0.930180	414.465740
1	SunlightTime/daylength	0.889645	444.685083
2	sunlightTime	0.886981	450.019563

In [31]:
```
col_to_drop = "sunlightTime"
w1_x_train2 = w1_x_train.loc[:, ~w1_x_train.columns.str.startswith(col_to_drop)]
w1_x_test2 = w1_x_test.loc[:, ~w1_x_test.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w1_x_train2)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  sunlightTime
```

Out[31]:

	feature	VIF
0	GHI	2.767105e+00
1	temp	4.327978e+00
2	pressure	1.356490e+00
3	humidity	3.318358e+00
4	wind_speed	1.340053e+00
5	clouds_all	1.082679e+00
6	isSun	4.094321e+00
7	dayLength	2.198866e+00
8	SunlightTime/daylength	4.056070e+00
9	weather_type	1.704300e+06
10	hour	1.378663e+00
11	month	1.680111e+00
12	capture_day	1.005944e+00
13	capture_year	1.077129e+00
14	capture_min	1.000917e+00

In [32]:
```
col_to_drop = "weather_type"
w1_x_train3 = w1_x_train2.loc[:, ~w1_x_train2.columns.str.startswith(col_to_drop)]
w1_x_test3 = w1_x_test2.loc[:, ~w1_x_test2.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w1_x_train3)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  weather_type
```

Out[32]:

	feature	VIF
0	GHI	4.046593
1	temp	9.908076
2	pressure	20325.564825
3	humidity	54.412911
4	wind_speed	9.212725

	feature	VIF
5	clouds_all	1.733272
6	isSun	7.966441
7	dayLength	53.884515
8	SunlightTime/daylength	6.192772
9	hour	4.824414
10	month	8.484728
11	capture_day	3.121215
12	capture_year	21290.147311
13	capture_min	2.808113

In [33]:
```
col_to_drop = "pressure"
w1_x_train4 = w1_x_train3.loc[:, ~w1_x_train3.columns.str.startswith(col_to_drop)]
w1_x_test4 = w1_x_test3.loc[:, ~w1_x_test3.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w1_x_train4)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  pressure
```

Out[33]:

	feature	VIF
0	GHI	4.034800
1	temp	9.368769
2	humidity	53.062123
3	wind_speed	8.551137
4	clouds_all	1.705297
5	isSun	7.965252
6	dayLength	53.791532
7	SunlightTime/daylength	6.192261
8	hour	4.817843
9	month	8.461388
10	capture_day	3.121019
11	capture_year	153.919947
12	capture_min	2.808105

In [34]:
```
col_to_drop = "capture_year"
w1_x_train5 = w1_x_train4.loc[:, ~w1_x_train4.columns.str.startswith(col_to_drop)]
w1_x_test5 = w1_x_test4.loc[:, ~w1_x_test4.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w1_x_train5)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  capture_year
```

Out[34]:

	feature	VIF
0	GHI	3.568708

	feature	VIF
1	temp	8.764006
2	humidity	26.586704
3	wind_speed	5.901301
4	clouds_all	1.678463
5	isSun	7.786087
6	dayLength	32.838772
7	SunlightTime/daylength	5.181837
8	hour	4.169868
9	month	8.262512
10	capture_day	3.100044
11	capture_min	2.770561

In [35]:
```
col_to_drop = "humidity"
w1_x_train6 = w1_x_train5.loc[:, ~w1_x_train5.columns.str.startswith(col_to_drop)]
w1_x_test6 = w1_x_test5.loc[:, ~w1_x_test5.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w1_x_train6)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  humidity
```

Out[35]:

	feature	VIF
0	GHI	3.418600
1	temp	5.464990
2	wind_speed	5.829997
3	clouds_all	1.633680
4	isSun	7.539794
5	dayLength	13.669835
6	SunlightTime/daylength	4.877015
7	hour	4.145389
8	month	5.738258
9	capture_day	3.076615
10	capture_min	2.742218

In [36]:
```
col_to_drop = "dayLength"
w1_x_train7 = w1_x_train6.loc[:, ~w1_x_train6.columns.str.startswith(col_to_drop)]
w1_x_test7 = w1_x_test6.loc[:, ~w1_x_test6.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w1_x_train7)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  dayLength
```

Out[36]:

	feature	VIF
0	GHI	3.411620

	feature	VIF
1	temp	4.606136
2	wind_speed	5.167501
3	clouds_all	1.608043
4	isSun	6.813214
5	SunlightTime/daylength	4.423742
6	hour	3.658416
7	month	5.566191
8	capture_day	2.821845
9	capture_min	2.552071

In [37]:
```
col_to_drop = "isSun"
w1_x_train8 = w1_x_train7.loc[:, ~w1_x_train7.columns.str.startswith(col_to_drop)]
w1_x_test8 = w1_x_test7.loc[:, ~w1_x_test7.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w1_x_train8)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  isSun
```

Out[37]:

	feature	VIF
0	GHI	2.495252
1	temp	4.540626
2	wind_speed	5.064121
3	clouds_all	1.603695
4	SunlightTime/daylength	2.470743
5	hour	3.378905
6	month	5.336118
7	capture_day	2.797302
8	capture_min	2.523124

In [38]:
```
w1_olsmodel2 = sm.OLS(w1_y_train, w1_x_train8).fit()
print(w1_olsmodel2.summary())
```

```
                                 OLS Regression Results
================================================================================================
Dep. Variable:       Energy delta[Wh]   R-squared (uncentered):                   0.908
Model:                           OLS   Adj. R-squared (uncentered):              0.908
Method:                Least Squares   F-statistic:                          2.156e+04
Date:               Sat, 01 Jul 2023   Prob (F-statistic):                        0.00
Time:                       11:57:36   Log-Likelihood:                     -1.4847e+05
No. Observations:              19583   AIC:                                  2.970e+05
Df Residuals:                  19574   BIC:                                  2.970e+05
Df Model:                          9
Covariance Type:           nonrobust
================================================================================================
                               coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------------------------
GHI                         20.6587      0.069    300.062      0.000      20.524      20.794
temp                       -23.1792      0.528    -43.913      0.000     -24.214     -22.145
wind_speed                  20.3825      2.104      9.690      0.000      16.259      24.506
clouds_all                  -7.4789      1.079     -6.930      0.000      -9.594      -5.363
```

```
SunlightTime/daylength    80.3513     13.308      6.038      0.000      54.266     106.436
hour                       5.7038      0.451     12.643      0.000       4.820       6.588
month                      8.5891      1.248      6.883      0.000       6.143      11.035
capture_day               -4.8997      1.564     -3.132      0.002      -7.966      -1.833
capture_min               -0.5964      0.192     -3.104      0.002      -0.973      -0.220
==============================================================================
Omnibus:                     2200.764   Durbin-Watson:                   1.996
Prob(Omnibus):                  0.000   Jarque-Bera (JB):            21602.307
Skew:                           0.027   Prob(JB):                         0.00
Kurtosis:                       8.145   Cond. No.                         314.
==============================================================================

Notes:
[1] R² is computed without centering (uncentered) since the model does not contain a const
ant.
[2] Standard Errors assume that the covariance matrix of the errors is correctly specifie
d.
```

Dropping high P values

Dropping the columns whose p-value is >0.05(5%)

In [39]:
```
# initial list of columns
cols = w1_x_train8.columns.tolist()

# setting an initial max p-value
max_p_value = 1

while len(cols) > 0:
    # defining the train set
    x_train_aux = w1_x_train8[cols]

    # fitting the model
    model = sm.OLS(w1_y_train, x_train_aux).fit()

    # getting the p-values and the maximum p-value
    p_values = model.pvalues
    max_p_value = max(p_values)

    # name of the variable with maximum p-value
    feature_with_p_max = p_values.idxmax()

    if max_p_value > 0.05:
        cols.remove(feature_with_p_max)
    else:
        break

selected_features = cols
print(selected_features)
```

```
['GHI', 'temp', 'wind_speed', 'clouds_all', 'SunlightTime/daylength', 'hour', 'month', 'ca
pture_day', 'capture_min']
```

In [40]:
```
w1_x_train9= w1_x_train8[selected_features]
w1_x_test9= w1_x_test8[selected_features]
w1_olsmodel3 = sm.OLS(w1_y_train, w1_x_train9).fit()
print(w1_olsmodel3.summary())
```

```
                                 OLS Regression Results
================================================================================
Dep. Variable:       Energy delta[Wh]   R-squared (uncentered):                   0.908
Model:                           OLS   Adj. R-squared (uncentered):              0.908
Method:                Least Squares   F-statistic:                          2.156e+04
Date:               Sat, 01 Jul 2023   Prob (F-statistic):                        0.00
Time:                       11:57:37   Log-Likelihood:                     -1.4847e+05
No. Observations:              19583   AIC:                                  2.970e+05
                               19574   BIC:                                  2.970e+05
```

```
Df Model:                           9
Covariance Type:            nonrobust
==============================================================================================
                             coef    std err          t      P>|t|      [0.025      0.975]
----------------------------------------------------------------------------------------------
GHI                       20.6587      0.069    300.062      0.000      20.524      20.794
temp                     -23.1792      0.528    -43.913      0.000     -24.214     -22.145
wind_speed                20.3825      2.104      9.690      0.000      16.259      24.506
clouds_all                -7.4789      1.079     -6.930      0.000      -9.594      -5.363
SunlightTime/daylength    80.3513     13.308      6.038      0.000      54.266     106.436
hour                       5.7038      0.451     12.643      0.000       4.820       6.588
month                      8.5891      1.248      6.883      0.000       6.143      11.035
capture_day               -4.8997      1.564     -3.132      0.002      -7.966      -1.833
capture_min               -0.5964      0.192     -3.104      0.002      -0.973      -0.220
==============================================================================
Omnibus:                     2200.764   Durbin-Watson:                   1.996
Prob(Omnibus):                  0.000   Jarque-Bera (JB):            21602.307
Skew:                           0.027   Prob(JB):                         0.00
Kurtosis:                       8.145   Cond. No.                         314.
==============================================================================

Notes:
[1] R² is computed without centering (uncentered) since the model does not contain a const
ant.
[2] Standard Errors assume that the covariance matrix of the errors is correctly specifie
d.
```

In [41]:
```
#Final equation
Equation = "Energy Delt(Wh) ="
print(Equation, end=" ")
for i in range(len(w1_x_train9.columns)):
    if i == 0:
        print(w1_olsmodel3.params[i], "+", end=" ")
    elif i != len(w1_x_train9.columns) - 1:
        print(
            "(",
            w1_olsmodel3.params[i],
            ")*(",
            w1_x_train9.columns[i],
            ")",
            "+",
            end=" ",
        )
    else:
        print("(", w1_olsmodel3.params[i], ")*(", w1_x_train9.columns[i], ")")
```

```
Energy Delt(Wh) = 20.658730134222044 + ( -23.17920344186072 )*( temp ) + ( 20.382494762087
585 )*( wind_speed ) + ( -7.478894879199357 )*( clouds_all ) + ( 80.35129676703073 )*( Sun
lightTime/daylength ) + ( 5.7038287297624235 )*( hour ) + ( 8.589130127256379 )*( month )
+ ( -4.8997061156686685 )*( capture_day ) + ( -0.5963918289935076 )*( capture_min )
```

2. Weather type 2

In [42]:
```
#Grouping all the data for weather type 2
w2_x=X.loc[data["weather_type"]==2]
w2_y=y.loc[data["weather_type"]==2]
```

In [43]:
```
#Exploring the data
w2_x.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 35428 entries, 144 to 196767
Data columns (total 18 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   GHI                     35428 non-null  float64
 1   temp                    35428 non-null  float64
                             35428 non-null  int64
```

```
 3   humidity                  35428 non-null  int64
 4   wind_speed                35428 non-null  float64
 5   rain_1h                   35428 non-null  float64
 6   snow_1h                   35428 non-null  float64
 7   clouds_all                35428 non-null  int64
 8   isSun                     35428 non-null  int64
 9   sunlightTime              35428 non-null  int64
 10  dayLength                 35428 non-null  int64
 11  SunlightTime/daylength    35428 non-null  float64
 12  weather_type              35428 non-null  int64
 13  hour                      35428 non-null  int64
 14  month                     35428 non-null  int64
 15  capture_day               35428 non-null  int64
 16  capture_year              35428 non-null  int64
 17  capture_min               35428 non-null  int64
dtypes: float64(6), int64(12)
memory usage: 5.1 MB
```

In [44]:
```
w2_x.describe()
```

Out[44]:

	GHI	temp	pressure	humidity	wind_speed	rain_1h	snow_1h	clouds_all	
count	35428.000000	35428.000000	35428.000000	35428.000000	35428.000000	35428.0	35428.0	35428.000000	3
mean	43.942291	11.401569	1016.858191	76.183019	3.763193	0.0	0.0	30.478717	
std	61.956739	8.259625	8.007208	15.994425	1.687911	0.0	0.0	11.614114	
min	0.000000	-15.400000	978.000000	27.000000	0.200000	0.0	0.0	11.000000	
25%	0.000000	5.100000	1012.000000	64.000000	2.400000	0.0	0.0	20.000000	
50%	2.600000	11.600000	1017.000000	79.000000	3.500000	0.0	0.0	30.000000	
75%	79.100000	17.400000	1022.000000	90.000000	4.800000	0.0	0.0	41.000000	
max	229.200000	34.800000	1046.000000	100.000000	13.400000	0.0	0.0	50.000000	

Observations

- The columns rain_1h and snow_1h are populated by 0s and will be dropped

In [45]:
```
#Dropping rain and snow columns as they only have 0 values for this subset
w2_x.drop(["rain_1h", "snow_1h"], axis=1, inplace=True)
w2_x.sample(10)
```

Out[45]:

	GHI	temp	pressure	humidity	wind_speed	clouds_all	isSun	sunlightTime	dayLength	SunlightTime/da
186377	192.0	16.9	1022	57	3.7	34	1	450	960	
162264	65.6	13.8	1026	86	4.7	21	1	135	855	
109169	0.0	5.6	1012	82	6.4	49	0	0	600	
124208	0.0	12.2	1015	76	2.0	11	0	0	975	
186774	144.4	26.7	1019	41	4.2	32	1	660	975	
29248	0.0	6.2	1023	91	1.9	26	0	0	540	
125042	197.4	24.6	1018	50	2.8	17	1	570	945	
116357	0.0	5.5	1009	94	3.4	26	0	0	915	
26578	0.0	7.4	1012	89	5.0	27	0	0	660	
119301	49.9	19.7	1012	52	4.6	25	1	900	1005	

In [46]:
```
# Building the linear model
w2_x=sm.add_constant(w2_x)
```

```
#Splitting the data into training and testing by the ratio 7:3
w2_x_train, w2_x_test, w2_y_train, w2_y_test = train_test_split(
    w2_x, w2_y, test_size=0.30, random_state=2
)
```

In [47]:
```
#Fitting the model
w2_model=sm.OLS(w2_y_train, w2_x_train).fit()
print(w2_model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:       Energy delta[Wh]   R-squared:                       0.860
Model:                            OLS   Adj. R-squared:                  0.860
Method:                 Least Squares   F-statistic:                 1.018e+04
Date:                Sat, 01 Jul 2023   Prob (F-statistic):               0.00
Time:                        11:57:39   Log-Likelihood:            -1.8732e+05
No. Observations:               24799   AIC:                         3.747e+05
Df Residuals:                   24783   BIC:                         3.748e+05
Df Model:                          15
Covariance Type:            nonrobust
================================================================================
                               coef    std err          t      P>|t|      [0.025      0.975]
--------------------------------------------------------------------------------
GHI                         19.9150      0.077    257.179      0.000      19.763      20.067
temp                        -4.5371      0.713     -6.367      0.000      -5.934      -3.140
pressure                     0.6496      0.406      1.600      0.110      -0.146       1.445
humidity                    -2.0286      0.322     -6.301      0.000      -2.660      -1.398
wind_speed                  -5.3188      2.036     -2.613      0.009      -9.309      -1.329
clouds_all                   0.5045      0.255      1.978      0.048       0.005       1.004
isSun                      -80.5025     12.013     -6.701      0.000    -104.049     -56.956
sunlightTime                -2.5153      0.053    -47.571      0.000      -2.619      -2.412
dayLength                   -0.3496      0.028    -12.502      0.000      -0.404      -0.295
SunlightTime/daylength    2033.7809     47.413     42.895      0.000    1940.849    2126.712
weather_type             1.414e+04   1855.246      7.622      0.000    1.05e+04    1.78e+04
hour                         1.9079      0.480      3.976      0.000       0.967       2.848
month                       -6.0030      1.260     -4.766      0.000      -8.472      -3.534
capture_day                  3.5750      1.429      2.502      0.012       0.774       6.376
capture_year               -14.0940      1.832     -7.692      0.000     -17.685     -10.503
capture_min                 -0.0620      0.175     -0.355      0.723      -0.404       0.280
==============================================================================
Omnibus:                     3615.193   Durbin-Watson:                   2.001
Prob(Omnibus):                  0.000   Jarque-Bera (JB):            57632.129
Skew:                          -0.004   Prob(JB):                         0.00
Kurtosis:                      10.468   Cond. No.                     1.53e+06
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specifie
d.
[2] The condition number is large, 1.53e+06. This might indicate that there are
strong multicollinearity or other numerical problems.
```

Multicolinearity

In [48]:
```
#Calculating the VIF of each column
checking_vif(w2_x_train)
```

Out[48]:

	feature	VIF
0	GHI	2.672413e+00
1	temp	4.015344e+00
2	pressure	1.220872e+00
3	humidity	3.081253e+00
	d_speed	1.362525e+00

	feature	VIF
5	clouds_all	1.020921e+00
6	isSun	4.183670e+00
7	sunlightTime	2.709430e+01
8	dayLength	2.986169e+00
9	SunlightTime/daylength	2.846699e+01
10	weather_type	1.601056e+06
11	hour	1.342400e+00
12	month	1.523963e+00
13	capture_day	1.003145e+00
14	capture_year	1.038535e+00
15	capture_min	1.000351e+00

In [49]:
```
col_list=["sunlightTime","weather_type"]
res = treating_multicollinearity(w2_x_train, w2_y_train, col_list)
res
```

Out[49]:

	col	Adj. R-squared after_dropping col	RMSE after dropping col
0	weather_type	0.900243	462.322523
1	sunlightTime	0.847532	482.404647

In [50]:
```
col_to_drop = "sunlightTime"
w2_x_train2 = w2_x_train.loc[:, ~w2_x_train.columns.str.startswith(col_to_drop)]
w2_x_test2 = w2_x_test.loc[:, ~w2_x_test.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w2_x_train2)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  sunlightTime
```

Out[50]:

	feature	VIF
0	GHI	2.608185e+00
1	temp	4.010841e+00
2	pressure	1.218904e+00
3	humidity	3.073517e+00
4	wind_speed	1.362345e+00
5	clouds_all	1.020917e+00
6	isSun	4.078367e+00
7	dayLength	2.544516e+00
8	SunlightTime/daylength	3.967191e+00
9	weather_type	1.601023e+06
10	hour	1.342060e+00
11	month	1.523527e+00
12	capture_day	1.003129e+00
13	ure_year	1.038531e+00

	feature	VIF
14	capture_min	1.000344e+00

In [51]:
```
col_to_drop = "weather_type"
w2_x_train3 = w2_x_train2.loc[:, ~w2_x_train2.columns.str.startswith(col_to_drop)]
w2_x_test3 = w2_x_test2.loc[:, ~w2_x_test2.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w2_x_train3)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  weather_type
```

Out[51]:

	feature	VIF
0	GHI	3.914512
1	temp	11.674517
2	pressure	19674.527944
3	humidity	72.508494
4	wind_speed	8.049776
5	clouds_all	8.010162
6	isSun	8.615619
7	dayLength	52.725234
8	SunlightTime/daylength	6.511458
9	hour	4.985220
10	month	9.015092
11	capture_day	3.054533
12	capture_year	20379.711333
13	capture_min	2.780913

In [52]:
```
col_to_drop = "pressure"
w2_x_train4 = w2_x_train3.loc[:, ~w2_x_train3.columns.str.startswith(col_to_drop)]
w2_x_test4 = w2_x_test3.loc[:, ~w2_x_test3.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w2_x_train4)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  pressure
```

Out[52]:

	feature	VIF
0	GHI	3.867549
1	temp	10.962486
2	humidity	71.656336
3	wind_speed	7.147229
4	clouds_all	7.990476
5	isSun	8.615042
6	dayLength	52.630357
7	SunlightTime/daylength	6.501496

	feature	VIF
8	hour	4.984741
9	month	8.975991
10	capture_day	3.053609
11	capture_year	173.097139
12	capture_min	2.780909

In [53]:
```
col_to_drop = "humidity"
w2_x_train5 = w2_x_train4.loc[:, ~w2_x_train4.columns.str.startswith(col_to_drop)]
w2_x_test5 = w2_x_test4.loc[:, ~w2_x_test4.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w2_x_train5)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  humidity
```

Out[53]:

	feature	VIF
0	GHI	3.270170
1	temp	10.071140
2	wind_speed	6.620990
3	clouds_all	7.950817
4	isSun	8.247542
5	dayLength	52.627619
6	SunlightTime/daylength	5.370923
7	hour	4.842621
8	month	8.407946
9	capture_day	3.053041
10	capture_year	76.847390
11	capture_min	2.780747

In [54]:
```
col_to_drop = "dayLength"
w2_x_train6 = w2_x_train5.loc[:, ~w2_x_train5.columns.str.startswith(col_to_drop)]
w2_x_test6 = w2_x_test5.loc[:, ~w2_x_test5.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w2_x_train6)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  dayLength
```

Out[54]:

	feature	VIF
0	GHI	3.261302
1	temp	4.983374
2	wind_speed	6.317844
3	clouds_all	7.950348
4	isSun	8.099803
5	SunlightTime/daylength	5.088005

	feature	VIF
6	hour	4.825463
7	month	6.807604
8	capture_day	3.052429
9	capture_year	26.609341
10	capture_min	2.780744

In [55]:
```
col_to_drop = "capture_year"
w2_x_train7 = w2_x_train6.loc[:, ~w2_x_train6.columns.str.startswith(col_to_drop)]
w2_x_test7 = w2_x_test6.loc[:, ~w2_x_test6.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w2_x_train7)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  capture_year
```

Out[55]:

	feature	VIF
0	GHI	3.260559
1	temp	4.921523
2	wind_speed	5.186648
3	clouds_all	6.306787
4	isSun	7.566040
5	SunlightTime/daylength	4.837316
6	hour	4.093958
7	month	5.850538
8	capture_day	2.779820
9	capture_min	2.597741

In [56]:
```
col_to_drop = "isSun"
w2_x_train8 = w2_x_train7.loc[:, ~w2_x_train7.columns.str.startswith(col_to_drop)]
w2_x_test8 = w2_x_test7.loc[:, ~w2_x_test7.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w2_x_train8)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  isSun
```

Out[56]:

	feature	VIF
0	GHI	2.549815
1	temp	4.912028
2	wind_speed	5.107191
3	clouds_all	6.131129
4	SunlightTime/daylength	2.515157
5	hour	3.671292
6	month	5.756705
7	capture_day	2.761685

	feature	VIF
8	capture_min	2.582123

In [57]:
```
col_to_drop = "clouds_all"
w2_x_train9 = w2_x_train8.loc[:, ~w2_x_train8.columns.str.startswith(col_to_drop)]
w2_x_test9 = w2_x_test8.loc[:, ~w2_x_test8.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w2_x_train9)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  clouds_all
```

Out[57]:

	feature	VIF
0	GHI	2.549191
1	temp	4.894996
2	wind_speed	4.397960
3	SunlightTime/daylength	2.514769
4	hour	3.538738
5	month	5.364265
6	capture_day	2.685423
7	capture_min	2.523300

In [58]:
```
#Printing model with columns left
w2_olsmodel2 = sm.OLS(w2_y_train,w2_x_train9).fit()
print(w2_olsmodel2.summary())
```

```
                                 OLS Regression Results
================================================================================================
Dep. Variable:       Energy delta[Wh]   R-squared (uncentered):                   0.887
Model:                            OLS   Adj. R-squared (uncentered):              0.886
Method:                 Least Squares   F-statistic:                          2.421e+04
Date:                Sat, 01 Jul 2023   Prob (F-statistic):                        0.00
Time:                        11:57:41   Log-Likelihood:                     -1.8896e+05
No. Observations:               24799   AIC:                                  3.779e+05
Df Residuals:                   24791   BIC:                                  3.780e+05
Df Model:                           8
Covariance Type:            nonrobust
================================================================================================
                               coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------------------------
GHI                         19.4736      0.066    295.390      0.000      19.344      19.603
temp                       -21.1629      0.493    -42.968      0.000     -22.128     -20.198
wind_speed                   9.9728      1.600      6.233      0.000       6.837      13.109
SunlightTime/daylength      50.5433     11.736      4.307      0.000      27.541      73.546
hour                         3.7429      0.432      8.671      0.000       2.897       4.589
month                       10.8231      1.032     10.489      0.000       8.801      12.846
capture_day                  3.1508      1.431      2.202      0.028       0.346       5.955
capture_min                 -0.0133      0.178     -0.075      0.940      -0.362       0.335
==============================================================================
Omnibus:                     3706.696   Durbin-Watson:                   2.007
Prob(Omnibus):                  0.000   Jarque-Bera (JB):            49418.709
Skew:                           0.263   Prob(JB):                         0.00
Kurtosis:                       9.896   Cond. No.                         293.
==============================================================================

Notes:
[1] R² is computed without centering (uncentered) since the model does not contain a const
ant.
```

```
[2] Standard Errors assume that the covariance matrix of the errors is correctly specifie
d.
```

Dropping high P values

Dropping the columns whose p-value is >0.05(5%)

In [59]:
```
# initial list of columns
cols = w2_x_train9.columns.tolist()

# setting an initial max p-value
max_p_value = 1

while len(cols) > 0:
    # defining the train set
    x_train_aux = w2_x_train[cols]

    # fitting the model
    model = sm.OLS(w2_y_train, x_train_aux).fit()

    # getting the p-values and the maximum p-value
    p_values = model.pvalues
    max_p_value = max(p_values)

    # name of the variable with maximum p-value
    feature_with_p_max = p_values.idxmax()

    if max_p_value > 0.05:
        cols.remove(feature_with_p_max)
    else:
        break

selected_features = cols
print(selected_features)
```

```
['GHI', 'temp', 'wind_speed', 'SunlightTime/daylength', 'hour', 'month', 'capture_day']
```

In [60]:
```
w2_x_train10= w2_x_train9[selected_features]
w2_x_test10= w2_x_test9[selected_features]
w2_olsmodel3 = sm.OLS(w2_y_train, w2_x_train10).fit()
print(w2_olsmodel3.summary())
```

```
                                 OLS Regression Results
================================================================================================
Dep. Variable:       Energy delta[Wh]   R-squared (uncentered):                   0.887
Model:                           OLS   Adj. R-squared (uncentered):              0.886
Method:                Least Squares   F-statistic:                          2.767e+04
Date:               Sat, 01 Jul 2023   Prob (F-statistic):                        0.00
Time:                       11:57:41   Log-Likelihood:                     -1.8896e+05
No. Observations:              24799   AIC:                                  3.779e+05
Df Residuals:                  24792   BIC:                                  3.780e+05
Df Model:                          7
Covariance Type:           nonrobust
================================================================================================
                               coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------------------------
GHI                         19.4734      0.066    295.610      0.000      19.344      19.603
temp                       -21.1645      0.492    -43.016      0.000     -22.129     -20.200
wind_speed                   9.9482      1.565      6.355      0.000       6.880      13.016
SunlightTime/daylength      50.5672     11.731      4.311      0.000      27.574      73.561
hour                         3.7386      0.428      8.739      0.000       2.900       4.577
month                       10.8108      1.019     10.612      0.000       8.814      12.808
capture_day                  3.1377      1.420      2.210      0.027       0.355       5.921
==============================================================================
Omnibus:                     3706.840   Durbin-Watson:                   2.007
Prob(Omnibus):                  0.000   Jarque-Bera (JB):            49418.323
                                0.263   Prob(JB):                         0.00
```

```
Kurtosis:                       9.896   Cond. No.                         288.
==============================================================================

Notes:
[1] R² is computed without centering (uncentered) since the model does not contain a const
ant.
[2] Standard Errors assume that the covariance matrix of the errors is correctly specifie
d.
```

In [61]:
```
#Final equation
Equation = "Energy Delt(Wh) ="
print(Equation, end=" ")
for i in range(len(w2_x_train10.columns)):
    if i == 0:
        print(w2_olsmodel3.params[i], "+", end=" ")
    elif i != len(w2_x_train10.columns) - 1:
        print(
            "(",
            w2_olsmodel3.params[i],
            ")*(",
            w2_x_train10.columns[i],
            ")",
            "+",
            end=" ",
        )
    else:
        print("(", w2_olsmodel3.params[i], ")*(", w2_x_train10.columns[i], ")")
```

```
Energy Delt(Wh) = 19.47343814414258 + ( -21.164538660250525 )*( temp ) + ( 9.9481507350478
06 )*( wind_speed ) + ( 50.567211106003484 )*( SunlightTime/daylength ) + ( 3.738630471481
294 )*( hour ) + ( 10.810843039333356 )*( month ) + ( 3.1376863014434813 )*( capture_day )
```

3. Weather type 3

In [62]:
```
#Grouping all the data for weather type 3
w3_x=X.loc[data["weather_type"]==3]
w3_y=y.loc[data["weather_type"]==3]
```

In [63]:
```
w3_x.describe()
```

Out[63]:

	GHI	temp	pressure	humidity	wind_speed	rain_1h	snow_1h	clouds_all	3
count	31660.000000	31660.000000	31660.000000	31660.000000	31660.000000	31660.0	31660.0	31660.000000	
mean	37.090553	10.317524	1015.866582	78.008465	3.868111	0.0	0.0	68.236766	
std	55.350809	8.322163	8.745162	15.358958	1.792632	0.0	0.0	9.994308	
min	0.000000	-14.200000	980.000000	26.000000	0.100000	0.0	0.0	51.000000	
25%	0.000000	3.800000	1011.000000	67.000000	2.500000	0.0	0.0	60.000000	
50%	3.800000	10.400000	1016.000000	82.000000	3.600000	0.0	0.0	69.000000	
75%	60.700000	16.600000	1021.000000	91.000000	5.000000	0.0	0.0	77.000000	
max	229.100000	34.500000	1047.000000	100.000000	14.200000	0.0	0.0	84.000000	

In [64]:
```
#Dropping rain and snow columns as they only have 0 values for this subset
w3_x.drop(["rain_1h", "snow_1h"], axis=1, inplace=True)
w3_x.sample(10)
```

Out[64]:

	GHI	temp	pressure	humidity	wind_speed	clouds_all	isSun	sunlightTime	dayLength	SunlightTime/da
154172	0.0	9.0	1020	86	2.0	82	0	0	990	
142675	0.0	-7.4	1003	95	2.5	78	0	0	540	

	GHI	temp	pressure	humidity	wind_speed	clouds_all	isSun	sunlightTime	dayLength	SunlightTime/da
111695	135.5	7.5	1022	42	7.6	59	1	405	705	
95964	17.2	9.8	1012	85	4.1	76	1	615	690	
165390	0.0	15.0	1017	96	1.7	57	0	0	720	
179895	0.0	-1.2	1026	86	1.2	72	0	0	675	
26863	0.0	10.8	1016	76	8.0	60	0	0	660	
159606	128.3	28.7	1011	56	3.0	57	1	645	960	
135336	0.0	9.4	1015	88	3.7	52	0	0	510	
19423	150.0	16.8	1005	86	3.7	79	1	300	960	

In [65]:
```
# Building the linear model
w3_x=sm.add_constant(w3_x)

#Splitting the data into training and testing by the ratio 7:3
w3_x_train, w3_x_test, w3_y_train, w3_y_test = train_test_split(
    w3_x, w3_y, test_size=0.30, random_state=3
)
```

In [66]:
```
#Fitting the model
w3_olsmodel=sm.OLS(w3_y_train, w3_x_train).fit()
print(w3_olsmodel.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:       Energy delta[Wh]   R-squared:                       0.845
Model:                            OLS   Adj. R-squared:                  0.845
Method:                 Least Squares   F-statistic:                     8049.
Date:                Sat, 01 Jul 2023   Prob (F-statistic):               0.00
Time:                        11:57:43   Log-Likelihood:            -1.6601e+05
No. Observations:               22162   AIC:                         3.321e+05
Df Residuals:                   22146   BIC:                         3.322e+05
Df Model:                          15
Covariance Type:            nonrobust
============================================================================================
                               coef    std err          t      P>|t|      [0.025      0.975]
--------------------------------------------------------------------------------------------
GHI                         19.5540      0.084    233.864      0.000      19.390      19.718
temp                        -1.3786      0.704     -1.958      0.050      -2.759       0.001
pressure                     1.8630      0.364      5.119      0.000       1.150       2.576
humidity                    -1.8439      0.326     -5.648      0.000      -2.484      -1.204
wind_speed                   4.1329      1.836      2.251      0.024       0.534       7.732
clouds_all                  -0.7969      0.292     -2.725      0.006      -1.370      -0.224
isSun                       -1.1152     11.801     -0.094      0.925     -24.247      22.016
sunlightTime                -1.6114      0.049    -32.717      0.000      -1.708      -1.515
dayLength                   -0.2640      0.027     -9.727      0.000      -0.317      -0.211
SunlightTime/daylength    1124.8631     42.190     26.662      0.000    1042.167    1207.559
weather_type              6443.8195   1220.635      5.279      0.000    4051.289    8836.350
hour                         0.9357      0.479      1.955      0.051      -0.002       1.874
month                       -3.5048      1.083     -3.236      0.001      -5.627      -1.382
capture_day                  1.2156      1.455      0.835      0.404      -1.637       4.068
capture_year               -10.3120      1.802     -5.724      0.000     -13.843      -6.781
capture_min                  0.0422      0.173      0.243      0.808      -0.298       0.382
==============================================================================
Omnibus:                     4075.337   Durbin-Watson:                   2.029
Prob(Omnibus):                  0.000   Jarque-Bera (JB):            75922.881
Skew:                           0.355   Prob(JB):                         0.00
Kurtosis:                      12.040   Cond. No.                     1.01e+06
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specifie
```

```
[2] The condition number is large, 1.01e+06. This might indicate that there are
strong multicollinearity or other numerical problems.
```

Multicolinearity

In [67]:
```
#Calculating the VIF of each column
checking_vif(w3_x_train)
```

Out[67]:

	feature	VIF
0	GHI	2.517055e+00
1	temp	4.028519e+00
2	pressure	1.192634e+00
3	humidity	2.969291e+00
4	wind_speed	1.287984e+00
5	clouds_all	1.006748e+00
6	isSun	4.082107e+00
7	sunlightTime	2.296298e+01
8	dayLength	3.319685e+00
9	SunlightTime/daylength	2.363334e+01
10	weather_type	1.580650e+06
11	hour	1.288625e+00
12	month	1.463895e+00
13	capture_day	1.006005e+00
14	capture_year	1.044142e+00
15	capture_min	1.000323e+00

In [68]:
```
col_list=["sunlightTime","weather_type"]
res = treating_multicollinearity(w3_x_train, w3_y_train, col_list)
res
```

Out[68]:

	col	Adj. R-squared after_dropping col	RMSE after dropping col
0	weather_type	0.886603	433.867203
1	sunlightTime	0.837402	443.949613

In [69]:
```
col_to_drop = "sunlightTime"
w3_x_train2 = w3_x_train.loc[:, ~w3_x_train.columns.str.startswith(col_to_drop)]
w3_x_test2 = w3_x_test.loc[:, ~w3_x_test.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w3_x_train2)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  sunlightTime
```

Out[69]:

	feature	VIF
0	GHI	2.421583e+00
1	temp	4.021308e+00
2	pressure	1.191059e+00
3	humidity	2.933069e+00

	feature	VIF
4	wind_speed	1.287658e+00
5	clouds_all	1.005836e+00
6	isSun	3.954622e+00
7	dayLength	2.853708e+00
8	SunlightTime/daylength	4.005973e+00
9	weather_type	1.580639e+06
10	hour	1.288338e+00
11	month	1.463882e+00
12	capture_day	1.005258e+00
13	capture_year	1.044116e+00
14	capture_min	1.000321e+00

In [70]:
```
col_to_drop = "weather_type"
w3_x_train3 = w3_x_train2.loc[:, ~w3_x_train2.columns.str.startswith(col_to_drop)]
w3_x_test3 = w3_x_test2.loc[:, ~w3_x_test2.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w3_x_train3)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  weather_type
```

Out[70]:

	feature	VIF
0	GHI	3.513189
1	temp	10.245951
2	pressure	15900.942799
3	humidity	77.747926
4	wind_speed	7.208454
5	clouds_all	47.927661
6	isSun	8.528557
7	dayLength	46.334282
8	SunlightTime/daylength	6.815538
9	hour	4.857177
10	month	7.014054
11	capture_day	3.213632
12	capture_year	16518.021443
13	capture_min	2.785202

In [71]:
```
col_to_drop = "pressure"
w3_x_train4 = w3_x_train3.loc[:, ~w3_x_train3.columns.str.startswith(col_to_drop)]
w3_x_test4 = w3_x_test3.loc[:, ~w3_x_test3.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w3_x_train4)
print("VIF after dropping ", col_to_drop)
vif
```

VIF after dropping pressure

Out[71]:

	feature	VIF
0	GHI	3.487013
1	temp	9.705135
2	humidity	76.543509
3	wind_speed	6.431194
4	clouds_all	47.927475
5	isSun	8.527803
6	dayLength	46.070558
7	SunlightTime/daylength	6.814368
8	hour	4.856713
9	month	6.933047
10	capture_day	3.211922
11	capture_year	228.833411
12	capture_min	2.785197

In [72]:

```
col_to_drop = "capture_year"
w3_x_train5 = w3_x_train4.loc[:, ~w3_x_train4.columns.str.startswith(col_to_drop)]
w3_x_test5 = w3_x_test4.loc[:, ~w3_x_test4.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w3_x_train5)
print("VIF after dropping ", col_to_drop)
vif
```

VIF after dropping capture_year

Out[72]:

	feature	VIF
0	GHI	3.212679
1	temp	9.536708
2	humidity	37.633501
3	wind_speed	5.376133
4	clouds_all	36.555356
5	isSun	8.379237
6	dayLength	34.674342
7	SunlightTime/daylength	6.141809
8	hour	4.470512
9	month	6.769504
10	capture_day	3.172155
11	capture_min	2.760134

In [73]:

```
col_to_drop = "humidity"
w3_x_train6 = w3_x_train5.loc[:, ~w3_x_train5.columns.str.startswith(col_to_drop)]
w3_x_test6 = w3_x_test5.loc[:, ~w3_x_test5.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w3_x_train6)
```

```
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  humidity
```

Out[73]:

	feature	VIF
0	GHI	2.921146
1	temp	7.599108
2	wind_speed	5.350787
3	clouds_all	25.710280
4	isSun	8.094220
5	dayLength	27.514683
6	SunlightTime/daylength	5.660482
7	hour	4.461752
8	month	5.413779
9	capture_day	3.132293
10	capture_min	2.739424

In [74]:

```
col_to_drop = "dayLength"
w3_x_train7 = w3_x_train6.loc[:, ~w3_x_train6.columns.str.startswith(col_to_drop)]
w3_x_test7 = w3_x_test6.loc[:, ~w3_x_test6.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w3_x_train7)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  dayLength
```

Out[74]:

	feature	VIF
0	GHI	2.920645
1	temp	4.174340
2	wind_speed	5.347564
3	clouds_all	13.527330
4	isSun	7.769202
5	SunlightTime/daylength	5.326998
6	hour	4.405479
7	month	5.140782
8	capture_day	3.100632
9	capture_min	2.709684

In [75]:

```
col_to_drop = "clouds_all"
w3_x_train8 = w3_x_train7.loc[:, ~w3_x_train7.columns.str.startswith(col_to_drop)]
w3_x_test8 = w3_x_test7.loc[:, ~w3_x_test7.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w3_x_train8)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  clouds_all
```

Out[75]:

	feature	VIF
0	GHI	2.915956
1	temp	4.161147
2	wind_speed	4.306300
3	isSun	7.250740
4	SunlightTime/daylength	5.104550
5	hour	3.668965
6	month	4.486570
7	capture_day	2.804746
8	capture_min	2.531860

In [76]:
```
col_to_drop = "isSun"
w3_x_train9 = w3_x_train8.loc[:, ~w3_x_train8.columns.str.startswith(col_to_drop)]
w3_x_test9 = w3_x_test8.loc[:, ~w3_x_test8.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w3_x_train9)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  isSun
```

Out[76]:

	feature	VIF
0	GHI	2.461326
1	temp	4.160231
2	wind_speed	4.134276
3	SunlightTime/daylength	2.484688
4	hour	3.405524
5	month	4.391494
6	capture_day	2.762148
7	capture_min	2.507348

In [77]:
```
#Printing model with columns left
w3_olsmodel2 = sm.OLS(w3_y_train,w3_x_train9).fit()
print(w3_olsmodel2.summary())
```

```
                            OLS Regression Results
==============================================================================================
Dep. Variable:       Energy delta[Wh]   R-squared (uncentered):                   0.878
Model:                           OLS   Adj. R-squared (uncentered):              0.878
Method:                Least Squares   F-statistic:                          1.994e+04
Date:               Sat, 01 Jul 2023   Prob (F-statistic):                        0.00
Time:                       11:57:46   Log-Likelihood:                     -1.6684e+05
No. Observations:              22162   AIC:                                  3.337e+05
Df Residuals:                  22154   BIC:                                  3.338e+05
Df Model:                          8
Covariance Type:            nonrobust
==============================================================================================
                             coef    std err          t      P>|t|      [0.025      0.975]
----------------------------------------------------------------------------------------------
GHI                        19.2366      0.071    270.225      0.000      19.097      19.376
temp                      -14.6740      0.465    -31.546      0.000     -15.586     -13.762
wind_speed                  9.8562      1.440      6.845      0.000       7.034      12.678
SunlightTime/daylength     -6.6548     10.874     -0.612      0.541     -27.968      14.658
                            1.1234      0.416      2.702      0.007       0.309       1.938
```

```
month                 7.5376      0.886      8.509      0.000       5.801       9.274
capture_day           2.7762      1.398      1.986      0.047       0.036       5.517
capture_min           0.0109      0.171      0.064      0.949      -0.324       0.346
==============================================================================
Omnibus:                     4390.600   Durbin-Watson:                   2.024
Prob(Omnibus):                  0.000   Jarque-Bera (JB):            71970.724
Skew:                           0.497   Prob(JB):                         0.00
Kurtosis:                      11.772   Cond. No.                         249.
==============================================================================

Notes:
[1] R² is computed without centering (uncentered) since the model does not contain a const
ant.
[2] Standard Errors assume that the covariance matrix of the errors is correctly specifie
d.
```

Dropping high P values

Dropping the columns whose p-value is >0.05(5%)

In [78]:
```
# initial list of columns
cols = w3_x_train9.columns.tolist()

# setting an initial max p-value
max_p_value = 1

while len(cols) > 0:
    # defining the train set
    x_train_aux = w3_x_train[cols]

    # fitting the model
    model = sm.OLS(w3_y_train, x_train_aux).fit()

    # getting the p-values and the maximum p-value
    p_values = model.pvalues
    max_p_value = max(p_values)

    # name of the variable with maximum p-value
    feature_with_p_max = p_values.idxmax()

    if max_p_value > 0.05:
        cols.remove(feature_with_p_max)
    else:
        break

selected_features = cols
print(selected_features)
```

```
['GHI', 'temp', 'wind_speed', 'hour', 'month', 'capture_day']
```

In [79]:
```
w3_x_train10= w3_x_train9[selected_features]
w3_x_test10= w3_x_test9[selected_features]
w3_olsmodel3 = sm.OLS(w3_y_train, w3_x_train10).fit()
print(w3_olsmodel3.summary())
```

```
                                 OLS Regression Results
================================================================================
Dep. Variable:       Energy delta[Wh]   R-squared (uncentered):                   0.878
Model:                           OLS   Adj. R-squared (uncentered):              0.878
Method:                Least Squares   F-statistic:                          2.659e+04
Date:               Sat, 01 Jul 2023   Prob (F-statistic):                        0.00
Time:                       11:57:46   Log-Likelihood:                     -1.6684e+05
No. Observations:              22162   AIC:                                  3.337e+05
Df Residuals:                  22156   BIC:                                  3.337e+05
Df Model:                          6
Covariance Type:            nonrobust
==============================================================================
```

```
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
GHI           19.2208      0.066    289.869      0.000      19.091      19.351
temp         -14.7255      0.457    -32.213      0.000     -15.622     -13.829
wind_speed     9.7674      1.396      6.995      0.000       7.031      12.504
hour           1.0765      0.403      2.673      0.008       0.287       1.866
month          7.5798      0.871      8.702      0.000       5.872       9.287
capture_day    2.8002      1.384      2.023      0.043       0.088       5.513
==============================================================================
Omnibus:                     4382.249   Durbin-Watson:                   2.024
Prob(Omnibus):                  0.000   Jarque-Bera (JB):            71702.360
Skew:                           0.495   Prob(JB):                         0.00
Kurtosis:                      11.756   Cond. No.                         34.9
==============================================================================

Notes:
[1] R² is computed without centering (uncentered) since the model does not contain a const
ant.
[2] Standard Errors assume that the covariance matrix of the errors is correctly specifie
d.
```

In [80]:
```
#Final equation
Equation = "Energy Delt(Wh) ="
print(Equation, end=" ")
for i in range(len(w3_x_train10.columns)):
    if i == 0:
        print(w3_olsmodel3.params[i], "+", end=" ")
    elif i != len(w3_x_train10.columns) - 1:
        print(
            "(",
            w3_olsmodel3.params[i],
            ")*(",
            w3_x_train10.columns[i],
            ")",
            "+",
            end=" ",
        )
    else:
        print("(", w3_olsmodel3.params[i], ")*(", w3_x_train10.columns[i], ")")
```

```
Energy Delt(Wh) = 19.220831551017287 + ( -14.725510974635908 )*( temp ) + ( 9.767395935052
441 )*( wind_speed ) + ( 1.0765213887418312 )*( hour ) + ( 7.579813968272436 )*( month ) +
( 2.8001833387039925 )*( capture_day )
```

4. Weather type 4

In [81]:
```
#Grouping all the data for weather type 4
w4_x=X.loc[data["weather_type"]==4]
w4_y=y.loc[data["weather_type"]==4]
```

In [82]:
```
w4_x.describe()
```

Out[82]:

	GHI	temp	pressure	humidity	wind_speed	rain_1h	snow_1h	clouds_all	
count	73004.000000	73004.000000	73004.000000	73004.000000	73004.000000	73004.0	73004.0	73004.000000	7
mean	24.062094	8.637395	1015.095008	82.488576	3.904597	0.0	0.0	97.040217	
std	41.823949	7.565779	9.495561	14.239009	1.773260	0.0	0.0	4.200882	
min	0.000000	-16.600000	980.000000	27.000000	0.000000	0.0	0.0	85.000000	
25%	0.000000	2.700000	1009.000000	75.000000	2.600000	0.0	0.0	95.000000	
50%	0.000000	7.600000	1015.000000	87.000000	3.700000	0.0	0.0	99.000000	
75%	29.900000	14.200000	1021.000000	93.000000	5.000000	0.0	0.0	100.000000	
	000	35.100000	1047.000000	100.000000	13.600000	0.0	0.0	100.000000	

In [83]:
```
#Dropping rain and snow columns as they only have 0 values for this subset
w4_x.drop(["rain_1h", "snow_1h"], axis=1, inplace=True)
w4_x.sample(10)
```

Out[83]:

	GHI	temp	pressure	humidity	wind_speed	clouds_all	isSun	sunlightTime	dayLength	SunlightTime/da
119448	89.5	14.4	1000	65	4.9	98	1	225	1005	
107482	11.3	8.6	1003	77	6.7	100	1	480	540	
130374	0.0	10.2	997	100	1.7	100	0	0	720	
175289	0.0	6.4	1028	87	6.6	100	0	0	465	
19644	58.5	23.0	1013	68	3.8	85	1	720	945	
43458	21.1	14.9	1004	68	1.7	99	1	735	795	
33672	0.0	4.7	1028	89	5.7	86	0	0	450	
6191	90.8	5.1	1025	79	3.2	95	1	405	705	
4852	62.9	4.6	1018	63	5.3	100	1	435	630	
195115	181.3	28.4	1013	42	3.5	88	1	450	900	

In [84]:
```
# Building the linear model
w4_x=sm.add_constant(w4_x)

#Splitting the data into training and testing by the ratio 7:3
w4_x_train, w4_x_test, w4_y_train, w4_y_test = train_test_split(
    w4_x, w4_y, test_size=0.40, random_state=4
)
```

In [85]:
```
#Fitting the model
w4_olsmodel=sm.OLS(w4_y_train, w4_x_train).fit()
print(w4_olsmodel.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:       Energy delta[Wh]   R-squared:                       0.846
Model:                            OLS   Adj. R-squared:                  0.846
Method:                 Least Squares   F-statistic:                 1.609e+04
Date:                Sat, 01 Jul 2023   Prob (F-statistic):               0.00
Time:                        11:57:48   Log-Likelihood:            -3.1546e+05
No. Observations:               43802   AIC:                         6.310e+05
Df Residuals:                   43786   BIC:                         6.311e+05
Df Model:                          15
Covariance Type:            nonrobust
================================================================================================
                                   coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------------------------
GHI                            19.3358      0.056    345.728      0.000      19.226      19.445
temp                           -3.0767      0.378     -8.134      0.000      -3.818      -2.335
pressure                        1.1534      0.169      6.815      0.000       0.822       1.485
humidity                       -2.2688      0.169    -13.411      0.000      -2.600      -1.937
wind_speed                      1.9761      0.979      2.018      0.044       0.057       3.896
clouds_all                     -1.1703      0.378     -3.097      0.002      -1.911      -0.430
isSun                         -13.4847      6.051     -2.229      0.026     -25.344      -1.625
sunlightTime                   -0.7997      0.026    -30.183      0.000      -0.852      -0.748
dayLength                      -0.0684      0.015     -4.599      0.000      -0.097      -0.039
SunlightTime/daylength        414.7103     21.187     19.573      0.000     373.183     456.238
weather_type                 4473.0043    487.769      9.170      0.000    3516.968    5429.041
hour                            0.2793      0.251      1.115      0.265      -0.212       0.770
month                          -1.3269      0.490     -2.706      0.007      -2.288      -0.366
capture_day                     0.5777      0.791      0.731      0.465      -0.972       2.128
capture_year                   -9.2548      0.962     -9.618      0.000     -11.141      -7.369
capture_min                    -0.0866      0.092     -0.937      0.349      -0.268       0.095
```

```
==============================================================================
Omnibus:                    12666.771   Durbin-Watson:                   1.996
Prob(Omnibus):                  0.000   Jarque-Bera (JB):           334218.006
Skew:                           0.815   Prob(JB):                         0.00
Kurtosis:                      16.434   Cond. No.                     7.48e+05
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specifie
d.
[2] The condition number is large, 7.48e+05. This might indicate that there are
strong multicollinearity or other numerical problems.
```

Multicolinearity

In [86]:
```
#Calculating the VIF of each column
checking_vif(w4_x_train)
```

Out[86]:

	feature	VIF
0	GHI	2.279902e+00
1	temp	3.408283e+00
2	pressure	1.072967e+00
3	humidity	2.412866e+00
4	wind_speed	1.257639e+00
5	clouds_all	1.045828e+00
6	isSun	3.800198e+00
7	sunlightTime	1.932654e+01
8	dayLength	3.535066e+00
9	SunlightTime/daylength	1.958438e+01
10	weather_type	1.580538e+06
11	hour	1.224586e+00
12	month	1.394583e+00
13	capture_day	1.004835e+00
14	capture_year	1.026151e+00
15	capture_min	1.000079e+00

In [87]:
```
col_list=["sunlightTime","SunlightTime/daylength","weather_type"]
res = treating_multicollinearity(w4_x_train, w4_y_train, col_list)
res
```

Out[87]:

	col	Adj. R-squared after_dropping col	RMSE after dropping col
0	weather_type	0.876855	325.110135
1	SunlightTime/daylength	0.845084	326.216258
2	sunlightTime	0.843232	328.159886

In [88]:
```
col_to_drop = "sunlightTime"
w4_x_train2 = w4_x_train.loc[:, ~w4_x_train.columns.str.startswith(col_to_drop)]
w4_x_test2 = w4_x_test.loc[:, ~w4_x_test.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w4_x_train2)
```

```
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  sunlightTime
```

Out[88]:

	feature	VIF
0	GHI	2.203438e+00
1	temp	3.375398e+00
2	pressure	1.072960e+00
3	humidity	2.368176e+00
4	wind_speed	1.256483e+00
5	clouds_all	1.045682e+00
6	isSun	3.695020e+00
7	dayLength	3.168147e+00
8	SunlightTime/daylength	3.546063e+00
9	weather_type	1.580538e+06
10	hour	1.224572e+00
11	month	1.393712e+00
12	capture_day	1.004785e+00
13	capture_year	1.026149e+00
14	capture_min	1.000079e+00

In [89]:

```
col_to_drop = "weather_type"
w4_x_train3 = w4_x_train2.loc[:, ~w4_x_train2.columns.str.startswith(col_to_drop)]
w4_x_test3 = w4_x_test2.loc[:, ~w4_x_test2.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w4_x_train3)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  weather_type
```

Out[89]:

	feature	VIF
0	GHI	2.931851
1	temp	7.770237
2	pressure	12176.280108
3	humidity	80.893990
4	wind_speed	7.319346
5	clouds_all	559.191445
6	isSun	7.383904
7	dayLength	43.491975
8	SunlightTime/daylength	5.706930
9	hour	4.562444
10	month	5.628822
11	capture_day	3.385952
12	capture_year	13183.334588
13	ure_min	2.797934

In [90]:
```
col_to_drop = "pressure"
w4_x_train4 = w4_x_train3.loc[:, ~w4_x_train3.columns.str.startswith(col_to_drop)]
w4_x_test4 = w4_x_test3.loc[:, ~w4_x_test3.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w4_x_train4)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  pressure
```

Out[90]:

	feature	VIF
0	GHI	2.923920
1	temp	7.717335
2	humidity	80.796991
3	wind_speed	6.956556
4	clouds_all	559.146259
5	isSun	7.382540
6	dayLength	43.464259
7	SunlightTime/daylength	5.706881
8	hour	4.562180
9	month	5.626936
10	capture_day	3.385381
11	capture_year	711.695374
12	capture_min	2.797918

In [91]:
```
col_to_drop = "capture_year"
w4_x_train5 = w4_x_train4.loc[:, ~w4_x_train4.columns.str.startswith(col_to_drop)]
w4_x_test5 = w4_x_test4.loc[:, ~w4_x_test4.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w4_x_train5)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  capture_year
```

Out[91]:

	feature	VIF
0	GHI	2.779435
1	temp	7.505842
2	humidity	71.257080
3	wind_speed	6.688594
4	clouds_all	144.393044
5	isSun	7.289474
6	dayLength	37.710137
7	SunlightTime/daylength	5.619603
8	hour	4.473241
9	month	5.547150
10	capture_day	3.372241

	feature	VIF
11	capture_min	2.791814

In [92]:
```
col_to_drop = "clouds_all"
w4_x_train6 = w4_x_train5.loc[:, ~w4_x_train5.columns.str.startswith(col_to_drop)]
w4_x_test6 = w4_x_test5.loc[:, ~w4_x_test5.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w4_x_train6)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  clouds_all
```

Out[92]:

	feature	VIF
0	GHI	2.625561
1	temp	7.252924
2	humidity	24.620389
3	wind_speed	5.201604
4	isSun	7.144953
5	dayLength	27.210158
6	SunlightTime/daylength	5.181824
7	hour	4.213130
8	month	5.374761
9	capture_day	3.354562
10	capture_min	2.770204

In [93]:
```
col_to_drop = "humidity"
w4_x_train7 = w4_x_train6.loc[:, ~w4_x_train6.columns.str.startswith(col_to_drop)]
w4_x_test7 = w4_x_test6.loc[:, ~w4_x_test6.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w4_x_train7)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  humidity
```

Out[93]:

	feature	VIF
0	GHI	2.401685
1	temp	5.068661
2	wind_speed	4.853336
3	isSun	6.929048
4	dayLength	13.264817
5	SunlightTime/daylength	5.017899
6	hour	4.099152
7	month	3.699117
8	capture_day	3.227709
9	capture_min	2.705739

In [94]:
```
col_to_drop = "dayLength"
w4_x_train8 = w4_x_train7.loc[:, ~w4_x_train7.columns.str.startswith(col_to_drop)]
w4_x_test8 = w4_x_test7.loc[:, ~w4_x_test7.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w4_x_train8)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  dayLength
```

Out[94]:

	feature	VIF
0	GHI	2.400968
1	temp	3.296045
2	wind_speed	4.411447
3	isSun	6.305324
4	SunlightTime/daylength	4.698534
5	hour	3.562313
6	month	3.689470
7	capture_day	2.980027
8	capture_min	2.512847

In [95]:
```
col_to_drop = "isSun"
w4_x_train9 = w4_x_train8.loc[:, ~w4_x_train8.columns.str.startswith(col_to_drop)]
w4_x_test9 = w4_x_test8.loc[:, ~w4_x_test8.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w4_x_train9)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  isSun
```

Out[95]:

	feature	VIF
0	GHI	2.122374
1	temp	3.296037
2	wind_speed	4.248124
3	SunlightTime/daylength	2.198183
4	hour	3.324619
5	month	3.624339
6	capture_day	2.935792
7	capture_min	2.489086

In [96]:
```
#Printing model with columns left
w4_olsmodel2 = sm.OLS(w4_y_train,w4_x_train9).fit()
print(w4_olsmodel2.summary())
```

```
                                 OLS Regression Results
================================================================================
Dep. Variable:       Energy delta[Wh]   R-squared (uncentered):                   0.873
Model:                           OLS   Adj. R-squared (uncentered):              0.873
Method:                Least Squares   F-statistic:                          3.777e+04
Date:               Sat, 01 Jul 2023   Prob (F-statistic):                        0.00
Time:                       11:57:53   Log-Likelihood:                     -3.1612e+05
ns:                            43802   AIC:                                  6.322e+05
```

```
Df Residuals:                    43794   BIC:                                6.323e+05
Df Model:                            8
Covariance Type:            nonrobust
==============================================================================================
                               coef    std err          t      P>|t|      [0.025      0.975]
----------------------------------------------------------------------------------------------
GHI                         19.2292      0.047    405.343      0.000      19.136      19.322
temp                        -8.0620      0.249    -32.424      0.000      -8.549      -7.575
wind_speed                   6.9955      0.757      9.244      0.000       5.512       8.479
SunlightTime/daylength    -123.7865      5.678    -21.802      0.000    -134.915    -112.658
hour                         0.9206      0.217      4.241      0.000       0.495       1.346
month                        1.8248      0.398      4.586      0.000       1.045       2.605
capture_day                  0.6536      0.747      0.874      0.382      -0.811       2.118
capture_min                 -0.0694      0.088     -0.784      0.433      -0.243       0.104
==============================================================================
Omnibus:                    13141.585   Durbin-Watson:                   1.995
Prob(Omnibus):                  0.000   Jarque-Bera (JB):           327941.509
Skew:                           0.885   Prob(JB):                         0.00
Kurtosis:                      16.287   Cond. No.                          186.
==============================================================================

Notes:
[1] R² is computed without centering (uncentered) since the model does not contain a const
ant.
[2] Standard Errors assume that the covariance matrix of the errors is correctly specifie
d.
```

Dropping high P values

Dropping the columns whose p-value is >0.05(5%)

In [97]:
```
# initial list of columns
cols = w4_x_train9.columns.tolist()

# setting an initial max p-value
max_p_value = 1

while len(cols) > 0:
    # defining the train set
    x_train_aux = w4_x_train[cols]

    # fitting the model
    model = sm.OLS(w4_y_train, x_train_aux).fit()

    # getting the p-values and the maximum p-value
    p_values = model.pvalues
    max_p_value = max(p_values)

    # name of the variable with maximum p-value
    feature_with_p_max = p_values.idxmax()

    if max_p_value > 0.05:
        cols.remove(feature_with_p_max)
    else:
        break

selected_features = cols
print(selected_features)
```

```
['GHI', 'temp', 'wind_speed', 'SunlightTime/daylength', 'hour', 'month']
```

In [98]:
```
w4_x_train10= w4_x_train9[selected_features]
w4_x_test10= w4_x_test9[selected_features]
w4_olsmodel3 = sm.OLS(w4_y_train, w4_x_train10).fit()
print(w4_olsmodel3.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:       Energy delta[Wh]   R-squared (uncentered):                   0.873
Model:                           OLS   Adj. R-squared (uncentered):              0.873
Method:                Least Squares   F-statistic:                          5.036e+04
Date:               Sat, 01 Jul 2023   Prob (F-statistic):                        0.00
Time:                       11:57:54   Log-Likelihood:                     -3.1612e+05
No. Observations:              43802   AIC:                                  6.322e+05
Df Residuals:                  43796   BIC:                                  6.323e+05
Df Model:                          6
Covariance Type:            nonrobust
==============================================================================
                             coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
GHI                       19.2291      0.047    405.629      0.000      19.136      19.322
temp                      -8.0562      0.248    -32.488      0.000      -8.542      -7.570
wind_speed                 7.0621      0.693     10.197      0.000       5.705       8.420
SunlightTime/daylength  -123.8165      5.678    -21.808      0.000    -134.945    -112.688
hour                       0.9257      0.211      4.381      0.000       0.512       1.340
month                      1.8299      0.386      4.741      0.000       1.073       2.586
==============================================================================
Omnibus:                    13142.613   Durbin-Watson:                   1.995
Prob(Omnibus):                  0.000   Jarque-Bera (JB):           328036.304
Skew:                           0.885   Prob(JB):                         0.00
Kurtosis:                      16.289   Cond. No.                         178.
==============================================================================

Notes:
[1] R² is computed without centering (uncentered) since the model does not contain a const
ant.
[2] Standard Errors assume that the covariance matrix of the errors is correctly specifie
d.
```

In [99]:

```
#Final equation
Equation = "Energy Delt(Wh) ="
print(Equation, end=" ")
for i in range(len(w4_x_train10.columns)):
    if i == 0:
        print(w4_olsmodel3.params[i], "+", end=" ")
    elif i != len(w4_x_train10.columns) - 1:
        print(
            "(",
            w4_olsmodel3.params[i],
            ")*(",
            w4_x_train10.columns[i],
            ")",
            "+",
            end=" ",
        )
    else:
        print("(", w4_olsmodel3.params[i], ")*(", w4_x_train10.columns[i], ")")
```

```
Energy Delt(Wh) = 19.229071499816207 + ( -8.056203055759 )*( temp ) + ( 7.062093723472428
)*( wind_speed ) + ( -123.81648104791762 )*( SunlightTime/daylength ) + ( 0.92570763320602
98 )*( hour ) + ( 1.8298993421905922 )*( month )
```

In [ ]:

5. Weather type 5

In [100…

```
#Grouping all the data for weather type 5
w5_x=X.loc[data["weather_type"]==5]
w5_y=y.loc[data["weather_type"]==5]
```

In [101…

```
w5_x.describe()
```

Out[101…

	GHI	temp	pressure	humidity	wind_speed	rain_1h	snow_1h	clou
count	28708.000000	28708.00000	28708.000000	28708.000000	28708.000000	28708.000000	28708.000000	28708.0
mean	24.026731	9.63436	1008.091821	86.710464	4.873373	0.452629	0.048993	90.1
std	38.110953	6.75743	9.341614	10.487211	2.136564	0.598537	0.176803	18.2
min	0.000000	-9.80000	977.000000	39.000000	0.000000	0.000000	0.000000	2.0
25%	0.000000	4.20000	1002.000000	81.000000	3.300000	0.130000	0.000000	90.0
50%	5.500000	8.80000	1009.000000	90.000000	4.700000	0.250000	0.000000	100.0
75%	33.000000	15.00000	1015.000000	95.000000	6.200000	0.550000	0.000000	100.0
max	222.700000	32.00000	1037.000000	100.000000	14.300000	8.090000	2.820000	100.0

In [102…

```
# Building the linear model
w5_x=sm.add_constant(w5_x)

#Splitting the data into training and testing by the ratio 7:3
w5_x_train, w5_x_test, w5_y_train, w5_y_test = train_test_split(
    w5_x, w5_y, test_size=0.30, random_state=1
)
```

In [103…

```
#Fitting the model
w5_olsmodel=sm.OLS(w5_y_train, w5_x_train).fit()
print(w5_olsmodel.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:       Energy delta[Wh]   R-squared:                       0.788
Model:                            OLS   Adj. R-squared:                  0.787
Method:                 Least Squares   F-statistic:                     4379.
Date:                Sat, 01 Jul 2023   Prob (F-statistic):               0.00
Time:                        11:57:55   Log-Likelihood:            -1.4554e+05
No. Observations:               20095   AIC:                         2.911e+05
Df Residuals:                   20077   BIC:                         2.912e+05
Df Model:                          17
Covariance Type:            nonrobust
==============================================================================
                             coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
GHI                       16.8036      0.092    182.415      0.000      16.623      16.984
temp                       2.2648      0.654      3.465      0.001       0.984       3.546
pressure                   0.2742      0.290      0.945      0.345      -0.295       0.843
humidity                  -4.3413      0.329    -13.200      0.000      -4.986      -3.697
wind_speed                -5.6781      1.259     -4.510      0.000      -8.146      -3.210
rain_1h                    0.5636      4.549      0.124      0.901      -8.352       9.479
snow_1h                    5.5845     15.019      0.372      0.710     -23.854      35.023
clouds_all                -2.5225      0.149    -16.875      0.000      -2.816      -2.230
isSun                      7.1633      9.546      0.750      0.453     -11.547      25.874
sunlightTime              -0.4370      0.041    -10.728      0.000      -0.517      -0.357
dayLength                 -0.0775      0.023     -3.400      0.001      -0.122      -0.033
SunlightTime/daylength   136.8189     33.597      4.072      0.000      70.966     202.671
weather_type            3585.2087    586.773      6.110      0.000    2435.086    4735.331
hour                      -0.6400      0.416     -1.537      0.124      -1.456       0.176
month                     -0.1162      0.799     -0.145      0.884      -1.683       1.451
capture_day                1.3973      1.221      1.144      0.252      -0.996       3.791
capture_year              -8.6671      1.450     -5.979      0.000     -11.508      -5.826
capture_min               -0.1521      0.142     -1.068      0.286      -0.431       0.127
==============================================================================
Omnibus:                     6516.133   Durbin-Watson:                   2.034
Prob(Omnibus):                  0.000   Jarque-Bera (JB):           186749.953
Skew:                           0.958   Prob(JB):                         0.00
Kurtosis:                      17.811   Cond. No.                     5.88e+05
==============================================================================
```

```
Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specifie
d.
[2] The condition number is large, 5.88e+05. This might indicate that there are
strong multicollinearity or other numerical problems.
```

Multicolinearity

In [104…]
```
#Calculating the VIF of each column
checking_vif(w5_x_train)
```

Out[104…]

	feature	VIF
0	GHI	2.178346e+00
1	temp	3.458734e+00
2	pressure	1.274752e+00
3	humidity	2.104352e+00
4	wind_speed	1.266324e+00
5	rain_1h	1.282574e+00
6	snow_1h	1.242684e+00
7	clouds_all	1.311030e+00
8	isSun	3.909464e+00
9	sunlightTime	2.230067e+01
10	dayLength	3.547241e+00
11	SunlightTime/daylength	2.209805e+01
12	weather_type	1.511489e+06
13	hour	1.244287e+00
14	month	1.381674e+00
15	capture_day	1.009126e+00
16	capture_year	1.047270e+00
17	capture_min	1.000543e+00

In [105…]
```
#Original Model asjusted R value 0.842
col_list=["weather_type","SunlightTime/daylength","sunlightTime"]
res = treating_multicollinearity(w5_x_train, w5_y_train, col_list)
res
```

Out[105…]

	col	Adj. R-squared after_dropping col	RMSE after dropping col
0	weather_type	0.830734	338.589813
1	SunlightTime/daylength	0.787262	338.415136
2	sunlightTime	0.786219	339.243598

In [106…]
```
col_to_drop = "weather_type"
w5_x_train2 = w5_x_train.loc[:, ~w5_x_train.columns.str.startswith(col_to_drop)]
w5_x_test2 = w5_x_test.loc[:, ~w5_x_test.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w5_x_train2)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  weather_type
```

Out[106…

	feature	VIF
0	GHI	3.044811
1	temp	10.418291
2	pressure	14978.919744
3	humidity	144.105094
4	wind_speed	7.838130
5	rain_1h	2.022998
6	snow_1h	1.340596
7	clouds_all	33.204150
8	isSun	9.200344
9	sunlightTime	39.748655
10	dayLength	52.383830
11	SunlightTime/daylength	40.902593
12	hour	5.327448
13	month	5.391388
14	capture_day	3.319745
15	capture_year	15559.032707
16	capture_min	2.805606

In [107…

```
col_to_drop = "pressure"
w5_x_train3 = w5_x_train2.loc[:, ~w5_x_train2.columns.str.startswith(col_to_drop)]
w5_x_test3 = w5_x_test2.loc[:, ~w5_x_test2.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w5_x_train3)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  pressure
```

Out[107…

	feature	VIF
0	GHI	3.042256
1	temp	10.413586
2	humidity	143.255343
3	wind_speed	7.298891
4	rain_1h	2.000677
5	snow_1h	1.330673
6	clouds_all	32.903437
7	isSun	9.183077
8	sunlightTime	39.743207
9	dayLength	51.760994
10	SunlightTime/daylength	40.900526
11	hour	5.327441
12	month	5.357699

	feature	VIF
13	capture_day	3.313184
14	capture_year	239.179264
15	capture_min	2.805440

In [108…
```
col_to_drop = "humidity"
w5_x_train4 = w5_x_train3.loc[:, ~w5_x_train3.columns.str.startswith(col_to_drop)]
w5_x_test4 = w5_x_test3.loc[:, ~w5_x_test3.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w5_x_train4)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  humidity
```

Out[108…

	feature	VIF
0	GHI	2.747157
1	temp	10.146375
2	wind_speed	6.960886
3	rain_1h	1.888616
4	snow_1h	1.315189
5	clouds_all	31.258425
6	isSun	9.072975
7	sunlightTime	39.347490
8	dayLength	51.706110
9	SunlightTime/daylength	40.897223
10	hour	5.312737
11	month	5.305775
12	capture_day	3.310559
13	capture_year	98.800350
14	capture_min	2.805422

In [109…
```
col_to_drop = "capture_year"
w5_x_train5 = w5_x_train4.loc[:, ~w5_x_train4.columns.str.startswith(col_to_drop)]
w5_x_test5 = w5_x_test4.loc[:, ~w5_x_test4.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w5_x_train5)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  capture_year
```

Out[109…

	feature	VIF
0	GHI	2.675691
1	temp	9.605023
2	wind_speed	5.921218
3	rain_1h	1.872823
4	snow_1h	1.302432

	feature	VIF
5	clouds_all	19.219465
6	isSun	9.057332
7	sunlightTime	35.643829
8	dayLength	29.121924
9	SunlightTime/daylength	37.602040
10	hour	4.994475
11	month	4.698756
12	capture_day	3.244262
13	capture_min	2.757290

In [110…

```
col_to_drop = "sunlightTime"
w5_x_train6 = w5_x_train5.loc[:, ~w5_x_train5.columns.str.startswith(col_to_drop)]
w5_x_test6 = w5_x_test5.loc[:, ~w5_x_test5.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w5_x_train6)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  sunlightTime
```

Out[110…

	feature	VIF
0	GHI	2.585960
1	temp	9.299110
2	wind_speed	5.785733
3	rain_1h	1.868627
4	snow_1h	1.301459
5	clouds_all	18.136086
6	isSun	8.745728
7	dayLength	25.695411
8	SunlightTime/daylength	6.212066
9	hour	4.992099
10	month	4.600272
11	capture_day	3.229286
12	capture_min	2.751595

In [111…

```
col_to_drop = "dayLength"
w5_x_train7 = w5_x_train6.loc[:, ~w5_x_train6.columns.str.startswith(col_to_drop)]
w5_x_test7 = w5_x_test6.loc[:, ~w5_x_test6.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w5_x_train7)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  dayLength
```

Out[111…

	feature	VIF
0	GHI	2.478840

	feature	VIF
1	temp	5.392285
2	wind_speed	5.701474
3	rain_1h	1.868625
4	snow_1h	1.295822
5	clouds_all	12.983411
6	isSun	8.409689
7	SunlightTime/daylength	6.079994
8	hour	4.855998
9	month	4.451005
10	capture_day	3.185900
11	capture_min	2.714752

In [112…

```
col_to_drop = "clouds_all"
w5_x_train8 = w5_x_train7.loc[:, ~w5_x_train7.columns.str.startswith(col_to_drop)]
w5_x_test8 = w5_x_test7.loc[:, ~w5_x_test7.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w5_x_train8)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  clouds_all
```

Out[112…

	feature	VIF
0	GHI	2.352836
1	temp	5.260328
2	wind_speed	4.529485
3	rain_1h	1.848544
4	snow_1h	1.223996
5	isSun	7.660937
6	SunlightTime/daylength	5.767272
7	hour	4.274555
8	month	4.071324
9	capture_day	3.024053
10	capture_min	2.573801

In [113…

```
col_to_drop = "isSun"
w5_x_train9 = w5_x_train8.loc[:, ~w5_x_train8.columns.str.startswith(col_to_drop)]
w5_x_test9 = w5_x_test8.loc[:, ~w5_x_test8.columns.str.startswith(col_to_drop)]

# Check VIF now
vif = checking_vif(w5_x_train9)
print("VIF after dropping ", col_to_drop)
vif
```

```
VIF after dropping  isSun
```

Out[113…

	feature	VIF
0	GHI	2.142906

	feature	VIF
1	temp	5.220316
2	wind_speed	4.297625
3	rain_1h	1.839948
4	snow_1h	1.214432
5	SunlightTime/daylength	2.488360
6	hour	3.892138
7	month	4.026026
8	capture_day	3.005196
9	capture_min	2.553760

In [114…

```
#Printing model with columns left
w5_olsmodel2 = sm.OLS(w5_y_train,w5_x_train9).fit()
print(w5_olsmodel2.summary())
```

```
                                OLS Regression Results
==============================================================================================
Dep. Variable:       Energy delta[Wh]   R-squared (uncentered):                   0.825
Model:                           OLS   Adj. R-squared (uncentered):              0.825
Method:                Least Squares   F-statistic:                              9456.
Date:               Sat, 01 Jul 2023   Prob (F-statistic):                        0.00
Time:                       11:57:58   Log-Likelihood:                     -1.4591e+05
No. Observations:              20095   AIC:                                  2.918e+05
Df Residuals:                  20085   BIC:                                  2.919e+05
Df Model:                         10
Covariance Type:            nonrobust
==============================================================================================
                               coef    std err          t      P>|t|      [0.025      0.975]
----------------------------------------------------------------------------------------------
GHI                         17.3998      0.079    221.064      0.000      17.246      17.554
temp                         0.2285      0.471      0.486      0.627      -0.694       1.151
wind_speed                   0.1832      0.948      0.193      0.847      -1.675       2.041
rain_1h                    -27.3749      4.415     -6.200      0.000     -36.029     -18.721
snow_1h                    -45.3370     14.560     -3.114      0.002     -73.876     -16.798
SunlightTime/daylength    -144.1124      8.440    -17.075      0.000    -160.656    -127.569
hour                        -0.1372      0.363     -0.378      0.705      -0.848       0.573
month                        1.6407      0.693      2.367      0.018       0.282       3.000
capture_day                  1.6005      1.182      1.354      0.176      -0.717       3.918
capture_min                 -0.1809      0.138     -1.306      0.191      -0.452       0.091
==============================================================================
Omnibus:                     6931.040   Durbin-Watson:                   2.039
Prob(Omnibus):                  0.000   Jarque-Bera (JB):           200379.668
Skew:                           1.055   Prob(JB):                         0.00
Kurtosis:                      18.325   Cond. No.                         294.
==============================================================================

Notes:
[1] R² is computed without centering (uncentered) since the model does not contain a const
ant.
[2] Standard Errors assume that the covariance matrix of the errors is correctly specifie
d.
```

Dropping high P values

Dropping the columns whose p-value is >0.05(5%)

In [115…

```
# initial list of columns
cols = w5_x_train9.columns.tolist()

# setting an initial max p-value
```

```
max_p_value = 1

while len(cols) > 0:
    # defining the train set
    x_train_aux = w5_x_train[cols]

    # fitting the model
    model = sm.OLS(w5_y_train, x_train_aux).fit()

    # getting the p-values and the maximum p-value
    p_values = model.pvalues
    max_p_value = max(p_values)

    # name of the variable with maximum p-value
    feature_with_p_max = p_values.idxmax()

    if max_p_value > 0.05:
        cols.remove(feature_with_p_max)
    else:
        break

selected_features = cols
print(selected_features)
```

```
['GHI', 'rain_1h', 'snow_1h', 'SunlightTime/daylength', 'month']
```

In [116…

```
w5_x_train10= w5_x_train9[selected_features]
w5_x_test10= w5_x_test9[selected_features]
w5_olsmodel3 = sm.OLS(w5_y_train, w5_x_train10).fit()
print(w5_olsmodel3.summary())
```

```
                                 OLS Regression Results
================================================================================
Dep. Variable:       Energy delta[Wh]   R-squared (uncentered):                   0.825
Model:                            OLS   Adj. R-squared (uncentered):              0.825
Method:                 Least Squares   F-statistic:                          1.891e+04
Date:                Sat, 01 Jul 2023   Prob (F-statistic):                        0.00
Time:                        11:57:58   Log-Likelihood:                     -1.4591e+05
No. Observations:               20095   AIC:                                  2.918e+05
Df Residuals:                   20090   BIC:                                  2.919e+05
Df Model:                           5
Covariance Type:            nonrobust
================================================================================================
                                   coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------------------------
GHI                             17.4190      0.071    245.252      0.000      17.280      17.558
rain_1h                        -26.5890      4.059     -6.551      0.000     -34.545     -18.633
snow_1h                        -46.6458     13.465     -3.464      0.001     -73.039     -20.253
SunlightTime/daylength        -143.7852      7.881    -18.245      0.000    -159.233    -128.338
month                            1.8389      0.481      3.824      0.000       0.896       2.782
==============================================================================
Omnibus:                     6920.053   Durbin-Watson:                   2.039
Prob(Omnibus):                  0.000   Jarque-Bera (JB):           200160.611
Skew:                           1.052   Prob(JB):                         0.00
Kurtosis:                      18.318   Cond. No.                         252.
==============================================================================

Notes:
[1] R² is computed without centering (uncentered) since the model does not contain a const
ant.
[2] Standard Errors assume that the covariance matrix of the errors is correctly specifie
d.
```

In [117…

```
#Final equation
Equation = "Energy Delt(Wh) ="
print(Equation, end=" ")
for i in range(len(w5_x_train10.columns)):
    :
```

```
        print(w5_olsmodel3.params[i], "+", end=" ")
    elif i != len(w5_x_train10.columns) - 1:
        print(
            "(",
            w5_olsmodel3.params[i],
            ")*(",
            w5_x_train10.columns[i],
            ")",
            "+",
            end=" ",
        )
    else:
        print("(", w5_olsmodel3.params[i], ")*(", w5_x_train10.columns[i], ")")
```

```
Energy Delt(Wh) = 17.418998674911144 + ( -26.588993913965183 )*( rain_1h ) + ( -46.6458493
19975355 )*( snow_1h ) + ( -143.78517242885366 )*( SunlightTime/daylength ) + ( 1.83888017
08050364 )*( month )
```

Model Comparisons

In [118…

```
coef_pd=pd.DataFrame([w1_olsmodel3.params,w2_olsmodel3.params,w3_olsmodel3.params,w4_olsmo
coef_pd_final=coef_pd.T
coef_pd_final.columns=["Weather Type 1", "Weather Type 2","Weather Type 3","Weather Type 4
coef_pd_final
```

Out[118…

	Weather Type 1	Weather Type 2	Weather Type 3	Weather Type 4	Weather Type 5
GHI	20.658730	19.473438	19.220832	19.229071	17.418999
temp	-23.179203	-21.164539	-14.725511	-8.056203	NaN
wind_speed	20.382495	9.948151	9.767396	7.062094	NaN
clouds_all	-7.478895	NaN	NaN	NaN	NaN
SunlightTime/daylength	80.351297	50.567211	NaN	-123.816481	-143.785172
hour	5.703829	3.738630	1.076521	0.925708	NaN
month	8.589130	10.810843	7.579814	1.829899	1.838880
capture_day	-4.899706	3.137686	2.800183	NaN	NaN
capture_min	-0.596392	NaN	NaN	NaN	NaN
rain_1h	NaN	NaN	NaN	NaN	-26.588994
snow_1h	NaN	NaN	NaN	NaN	-46.645849

Observations

- Differnt Factors affect the energy consumption in the differnent weather types.
- There are 2 that are common across all 5 types:
    1. GHI
    2. Month
- In constrast: rain_1h and snow_1h are exclusive to only weather type 5
- Only one feature from the pairs with correlation greater than |0.68| was present in the final models of each weather type